

Глава 1. Основные понятия. Как обучить машину?	2
Зачем нужно машинное обучение?	2
Два типа машинного обучения	6
Обучение с учителем	6
Прогнозирование на основе примеров	6
Задача классификации	7
Задача регрессии	9
Обучение без учителя	9
Группировка новых данных	10
Снижение размерности	11
Параметрическое и непараметрическое обучение	11
Метод проб и ошибок или вычисления и вероятность	11
Параметрическое обучение с учителем	12
Метод проб и ошибок с использованием регуляторов	12
Параметрическое обучение без учителя	12
Кластеризация с использованием регуляторов	12
Непараметрическое обучение	13
Методы на основе вычислений	13
Обучающий, проверочный и тестовый наборы данных	13
Основные термины	14
Глава 2. Инструменты	15
Установка Python и Jupyter Notebook	15
Основы Jupyter Notebook	15
Основы NumPy	17
Основы Matplotlib	22
Основы Scikit-learn и визуализации данных	27
Глава 3. Обучение с учителем	32
Линейная регрессия	32
Шаг 1. Импорт библиотек и создание набора данных	33
Шаг 2. Разделение на обучающий и тестовый наборы	33
Шаг 3. Обучение модели	34
Шаг 4. Оценка на тестовом наборе данных	34
Шаг 5. Визуализация результата	35
Классификация	39
Метод К ближайших соседей	39
Глава 4. Обучение без учителя	46
Метод К-средних	46
СПИСОК ИСТОЧНИКОВ	55

Глава 1. Основные понятия. Как обучить машину?

Зачем нужно машинное обучение?

Некоторые задачи не получается решить традиционными методами программирования. Вам всем знаком случай, в котором машинное обучение оказалось необходимо — это фильтр спама в электронной почте. Давайте представим, как бы пришлось создать его стандартными техниками программирования.

Чтобы определить, является ли письмо спамом или не спамом, нужно проанализировать его состав. Чаще всего, нежелательные письма содержат определённые закономерности — слова или словосочетания вроде “бесплатный приз”, “кэшбэк” и тому подобные.

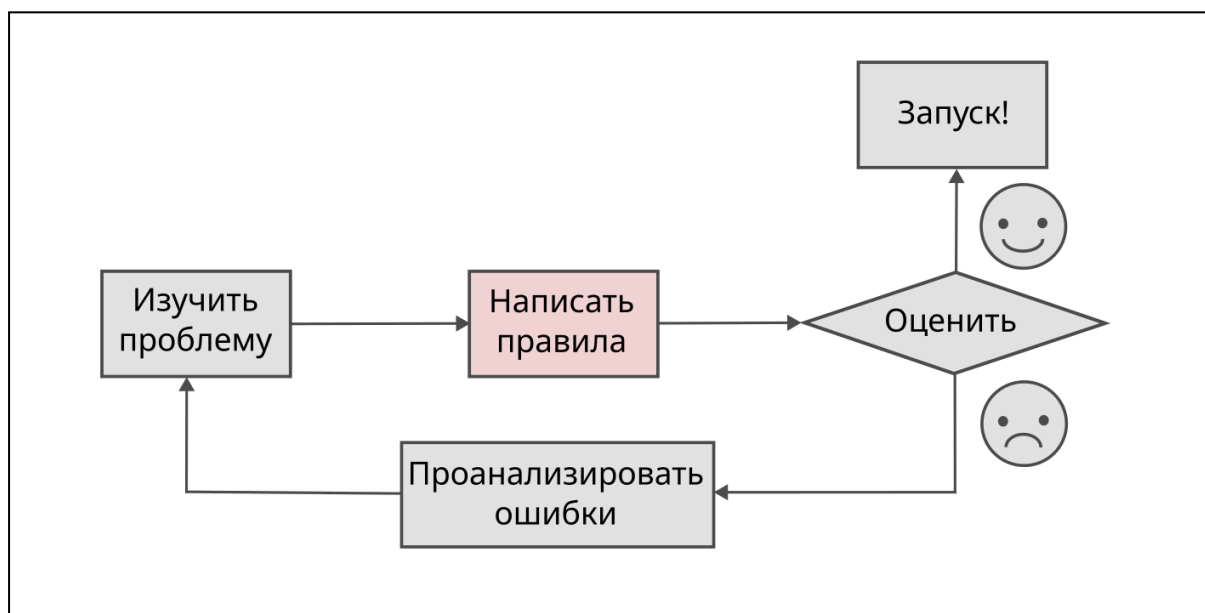


Рис. 1.1. Традиционный подход при создании спам-фильтра

Далее, пришлось бы написать алгоритм определения этих закономерностей, и письмо бы помечалось как спам или не спам, если в нём было замечено какое-то число этих паттернов. Так бы пришлось повторить несколько раз до того, как программа будет успешно работать.

Из-за того, что проблема необычна, поддерживать такой алгоритм сложно. Код быстро превратится в огромный список правил, который придётся изменять и дополнять.

Для решения подобных проблем используется машинное обучение. Спам-фильтр, использующий такой алгоритм, сам обучается тому, какие слова и словосочетания являются признаками спам-писем, находя слишком часто

появляющиеся закономерности в сравнении с обычными письмами. Такая программа короче, легче в настройке и управлении и, вероятно, более точна.

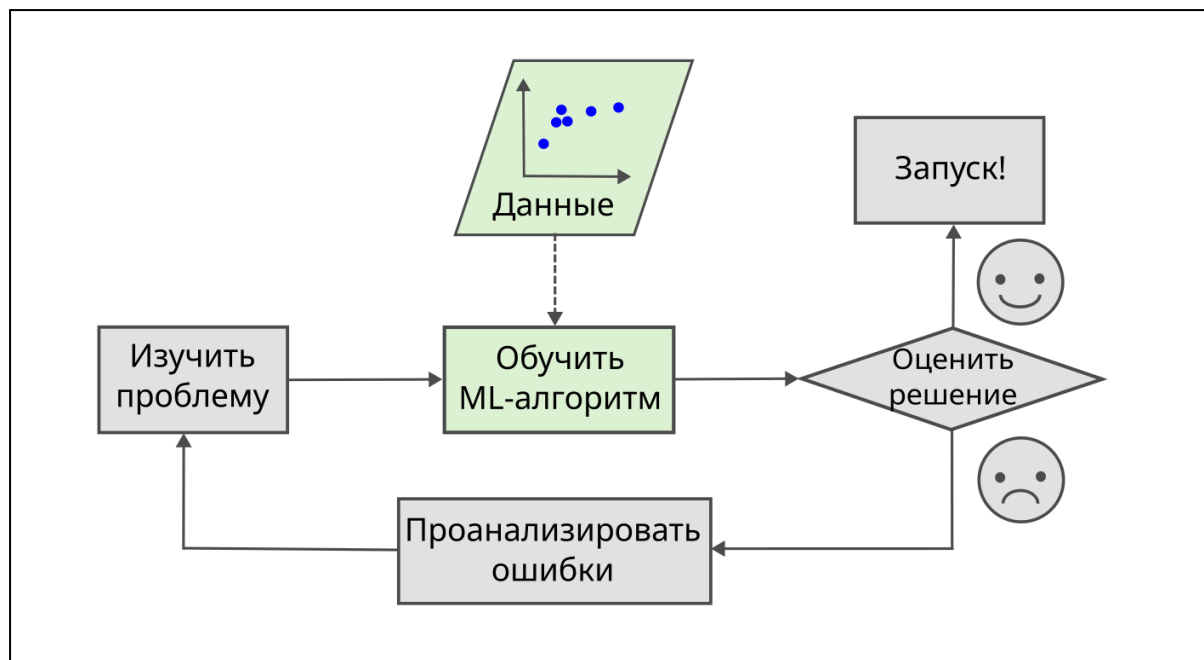


Рис. 1.2. Использование машинного обучения при создании спам-фильтра (ML — Machine Learning, с англ. — машинное обучение)

Более того, если спамеры заметят, что все их письма со словом “кэшбэк” блокируют, они начнут использовать “кешбек”, “кэшбек”, “кешбэк” и другие формы. В таком случае придётся вечно обновлять список правил, противодействуя новым способам обхода алгоритма.

Удивительным свойством машинного обучения является возможность адаптации. Алгоритм сам заметит, что вместо “кэшбэка” начал использоваться “кэшбек”, и будет блокировать такие письма без вмешательства создателя.

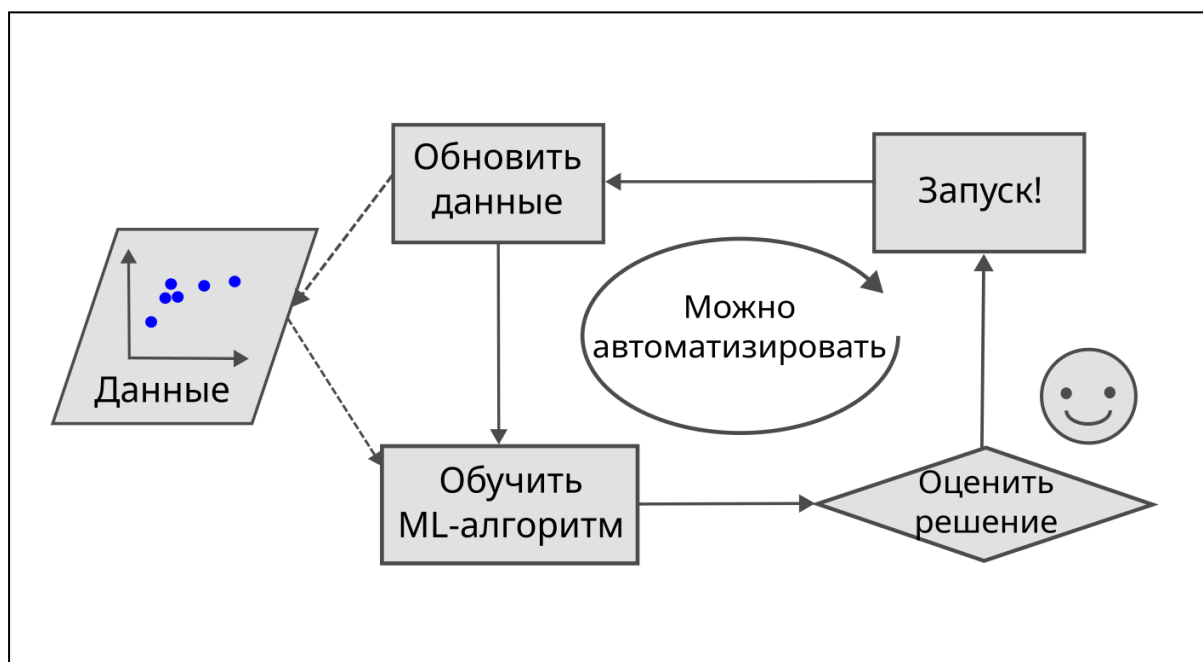


Рис. 1.3. Адаптация алгоритмов машинного обучения

Также машинное обучение полезно при решении слишком сложных для традиционного подхода проблем или проблем, не имеющих известного решения. Рассмотрим, например, распознавание речи: допустим, вы хотите начать с простого и написать программу, способную различать слова "один" и "два". Можно заметить, что слово "два" начинается с высокочастотного звука ("Д"), поэтому можно было бы вручную прописать в коде алгоритм, измеряющий интенсивность высокочастотного звука, и использовать его для различения "одного" и "двух". Очевидно, что такой подход не сработал бы для тысяч слов, сказанных миллионами разных людей на разных языках в шумных окружениях. Лучшим решением, по крайней мере на сегодняшний день, является самообучающийся алгоритм, обученный на примерах множества записей каждого конкретного слова.

Наконец, машинное обучение может помочь людям учиться (рис. 1.4): алгоритмы машинного обучения можно анализировать, чтобы понять, чему они научились (хотя для некоторых алгоритмов это может быть непросто). Например, после того как спам-фильтр обучен на достаточном количестве спама, его легко можно проанализировать, чтобы получить список слов и их сочетаний, которые алгоритм считает наилучшими признаками спама. Иногда это позволяет обнаружить неожиданные корреляции или новые тенденции и тем самым прийти к более глубокому пониманию проблемы. Применение методов МО для анализа больших объёмов данных помогает выявлять закономерности, которые не были очевидны с первого взгляда. Это называется *интеллектуальным анализом данных*.

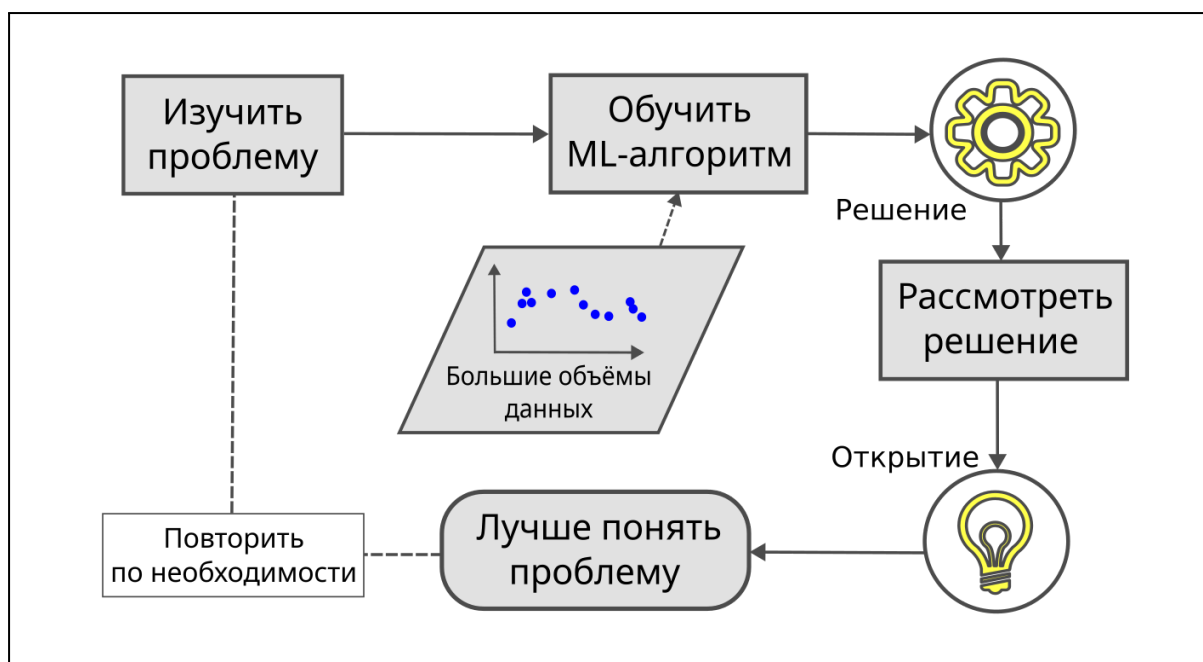


Рис. 1.4. Машинное обучение помогает в обучении людям

Подводя итог, машинное обучение отлично подходит для:

- Задач, существующие решения которых требуют длительной ручной настройки или длинных списков правил: один алгоритм машинного обучения зачастую способен упростить код и показать лучший результат.
- Сложных задач, для которых традиционный подход вовсе не даёт хорошего решения: лучшие методы машинного обучения способны найти решение там, где другие подходы бессильны.
- Изменчивых сред: система машинного обучения способна адаптироваться к новым данным.
- Получения ценных выводов о сложных проблемах и больших объёмах данных.»

Определение: *машинное обучение* – это область информатики, в которой машины учатся решать задачи *без непосредственного программирования их решений*. Проще говоря, машины наблюдают закономерности и пытаются повторить их тем или иным образом.

Область исследований, которая даёт компьютерам способность обучаться без непосредственного программирования.

— Артур Сэмюэль, основоположник машинного обучения, 1959

Машина будет пытаться имитировать закономерность разными способами в зависимости от вида её обучения.

Два типа машинного обучения

Существует множество способов обучить машину прогнозированию на основе данных, и для начала будут затронуты два наиболее популярных: обучение с учителем (supervised learning) и обучение без учителя (unsupervised learning). Слово “учитель” было выбрано как аналогия с процессом обучения с наставником. В обучении с учителем наставником выступает сам создатель алгоритма, показывая машине какие могут быть задачи и как их нужно решать. В обучении без учителя же машина делает всё сама по себе.

Обучение с учителем

Прогнозирование на основе примеров

Машинное обучение с учителем — метод преобразования одного набора данных в другой. Алгоритм получает на входе набор обучающих данных, среди которых есть примеры требуемого результата, называемые *метками*. Обучение с учителем, по сути, устанавливает взаимосвязь между входными данными и отмеченными примерами (рис. 1.5). Таким образом, обучение с учителем можно назвать “обучением по меткам”.

Машинное обучение с учителем лежит в основе прикладного (или же ограниченного) искусственного интеллекта. Такие алгоритмы используются, когда требуется на основе уже известных данных узнать что-то новое. Так машинное обучение помогает развиваться человечеству практически без ограничения, ведь позволяет получать новые знания из имеющейся информации при помощи логической оценки результатов.

Например, обучение с учителем используется в выше упомянутой фильтрации спама в электронной почте. Алгоритм обучается на основе пакета электронных писем, заранее отмеченных как “спам” или “не спам”. В процессе работы программа сравнивает содержимое письма с отмеченными данными и в результате выводит класс объекта, то есть категорию, к которой относится письмо.

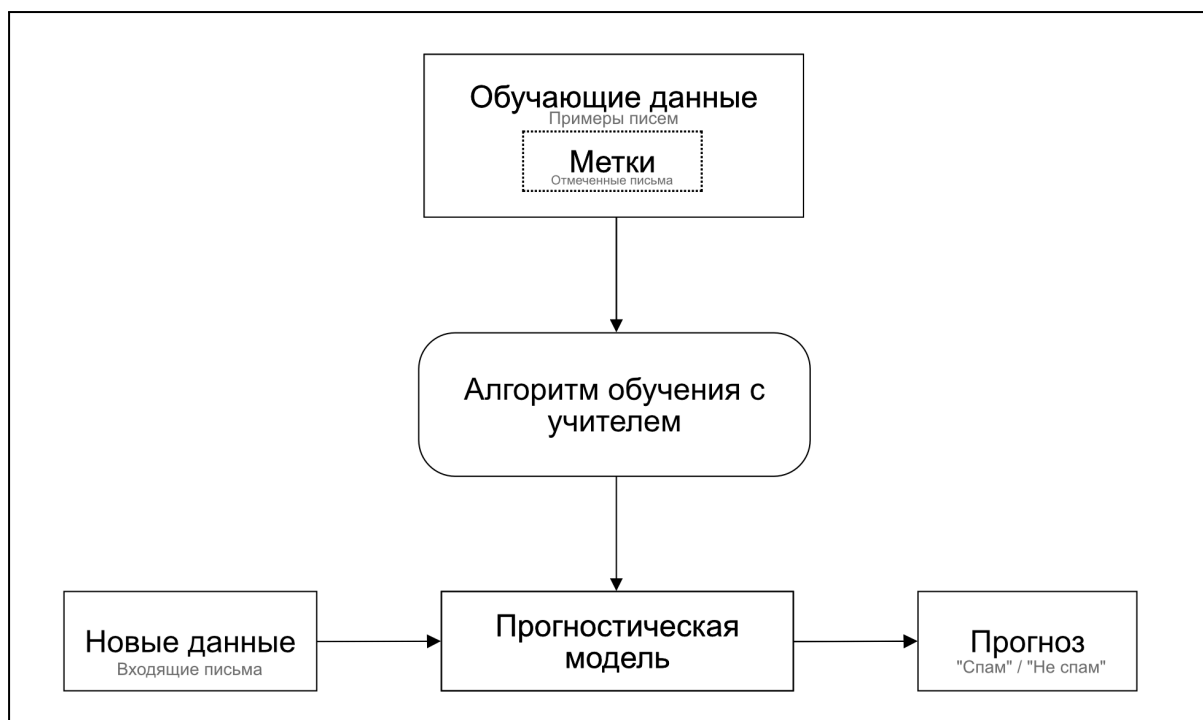


Рис. 1.5. Процесс обучения с учителем (серым обозначен пример спам-фильтра)

Такая задача по определению вида письма относится к *задаче классификации* (classification task) — одному из видов задач, где используется машинное обучение с учителем, и где результатом выполнения является дискретное значение (“спам”/”не спам”, “собака”/”кошка” и т.д). Также существует *задача регрессии* (regression task), где выводится некое действительное (вещественное) число (цена, температура, время).

Задача классификации

Целью классификации является прогнозирование того, к какому из отмеченных классов относится новая точка данных¹ на основе предыдущих наблюдений. В данном случае метки классов представляют собой дискретные неупорядоченные значения, которые служат примером для алгоритма при разделении точек данных на классы.

¹ Точка данных — единичный элемент набора данных. Часто результат алгоритмов машинного обучения выводится в виде графика, на котором этот элемент выглядит соответственно как точка.

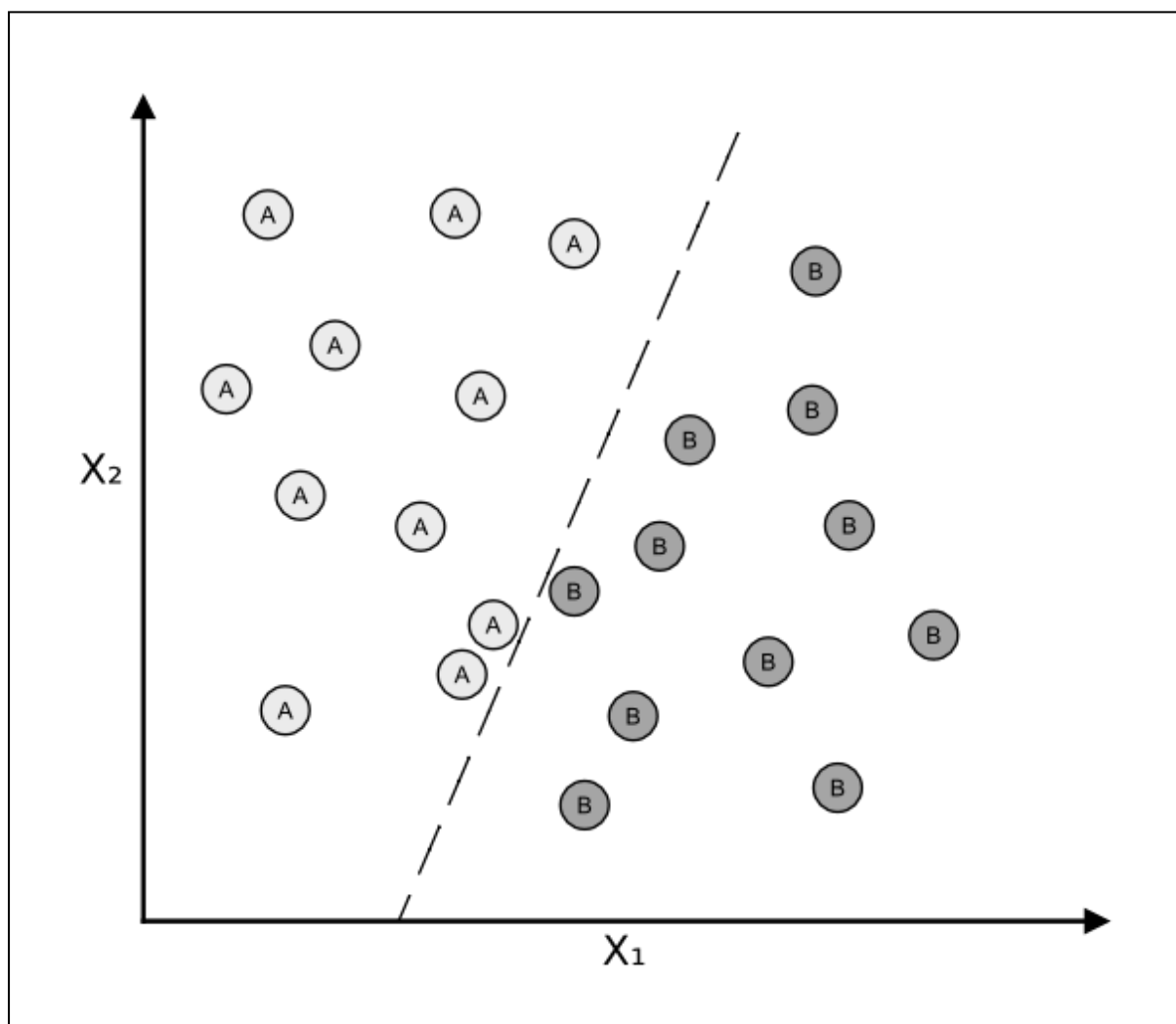


Рис. 1.6. Классификация данных

На рисунке 1.6 изображен пример классификации набора данных. В результате выполнения, алгоритм обозначил 11 обучающих примеров как класс А и 11 примеров как класс В. В данном случае набор данных является двумерным, то есть каждая точка данных определяется двумя признаками X_1 и X_2 по аналогии с тем, как в математике двумерные объекты обозначаются двумя координатными осями. Можно использовать этот результат, чтобы изучить правило, по которому новая точка данных будет относиться к той или иной категории. На графике им является пунктирная прямая, разделяющая точки одного класса от другого.

Стоит заметить, что количество возможных классов не ограничивается двумя. Важным примером *многоклассовой классификации* (multiclass classification) является распознавание рукописных символов. Для этой задачи можно использовать набор обучающих данных с несколькими примерами рукописной записи каждой буквы алфавита. В этом случае каждый символ (А, В, С и т.д.) будет являться отдельной меткой класса. Для английского алфавита такой набор будет 26-мерным. В дальнейшем, при вводе пользователем рукописного символа, прогностическая модель проанализирует его относительно всех 26 параметров и с определённой точностью

сделает прогноз введенной буквы. Однако если этот символ не входил в обучающий набор данных (например, это была цифра), то результат, очевидно, будет неверным.

Задача регрессии

Задача регрессии, также называемая *регрессионным анализом* - второй тип обучения с учителем, целью которого является прогнозирование непрерывных результатов. В регрессионном анализе, как и в классификации, входными данными (независимыми переменными) являются *признаки*, называемые *предикторами*, а результат выполнения — *целевая (зависимая) переменная*, являющаяся действительным числом. В задаче регрессии мы пытаемся найти между признаками и целевой переменной некую взаимосвязь, на основании которой можно будет предсказать результат для новых данных.

При помощи регрессионного анализа можно, например, предсказывать цены на недвижимость. Допустим, у нас есть набор данных о квартирах с несколькими признаками, такими как площадь, район, этаж и т.д. Если существует взаимосвязь между этими признаками и ценой, то ее можно будет увидеть на результатах регрессионной модели.

На рисунке 1.7 показана линейная регрессия. К данным, у которых известны признак X и целевая переменная Y , строится прямая, минимизирующая расстояние между точками и линией, чаще всего среднеквадратичное. Используя точку пересечения этой прямой с осью Y и угол ее наклона, можно предсказать целевую переменную для новой точки данных.

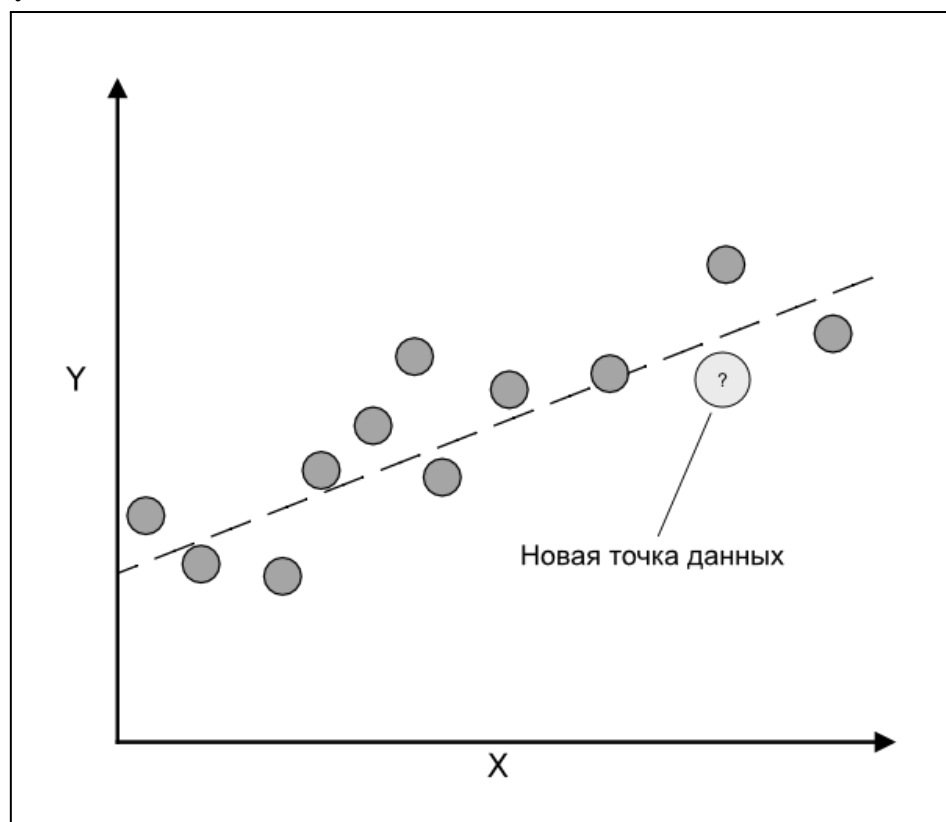


Рис. 1.7. Линейная регрессия

Обучение без учителя

Группировка новых данных

Обучение без учителя также преобразует один набор данных в другой. Однако заранее установленных меток у данных в этом методе нет. Алгоритм самостоятельно ищет закономерности в информации и выводит их. В основном, обучение без учителя работает путем *кластеризации*.

Кластеризация (clustering) — метод обнаружения закономерностей, который распределяет неупорядоченные данные по группам — кластерам (clusters), не зная заранее о принадлежности точек данных к определенным группам. Информация разбивается по степени схожести и различия между собой. Кластеризацию также часто называют *классификацией без учителя* (unsupervised classification).

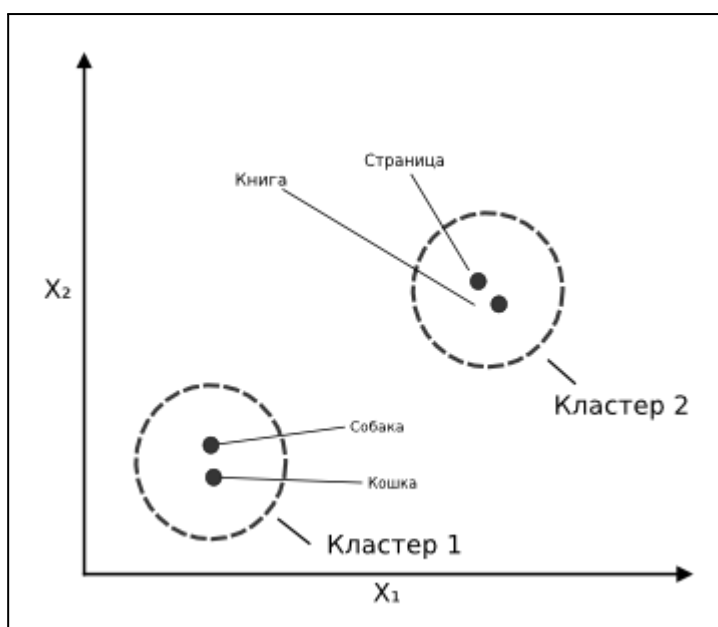


Рис 1.8. Кластеризация данных при обучении без учителя (X_1 и X_2 обозначены признаки кластеризации)

В роли меток в основном используются последовательные целые числа (то есть, когда образуется 10 кластеров, алгоритм присвоит каждому значение от 1 до 10).

Кластеры полезны как и сами по себе (например, маркетологам, которые разделяют так клиентов на основе их интересов), так и для создания алгоритма обучения с учителем. Также важной областью применения обучения без учителя является снижение размерности данных.

Снижение размерности

Часто мы работаем с данными высокой размерности, то есть со сложными наборами данных с большим количеством измерений и признаков. Использование таких наборов проблематично из-за ограничений в памяти и низкой эффективности вычислений алгоритмов машинного обучения. Чтобы сжать данные в наборе или удалить шум используется метод снижения размерности. Снижение размерности представляет исходные данные высокой размерности (то есть с большим количеством признаков) в пространстве меньшей размерности (рис. 1.9), сохраняя определенные характеристики. Существует несколько разных методов снижения размерности, и от них зависит, как и какая характеристика будет сохранена при переходе.

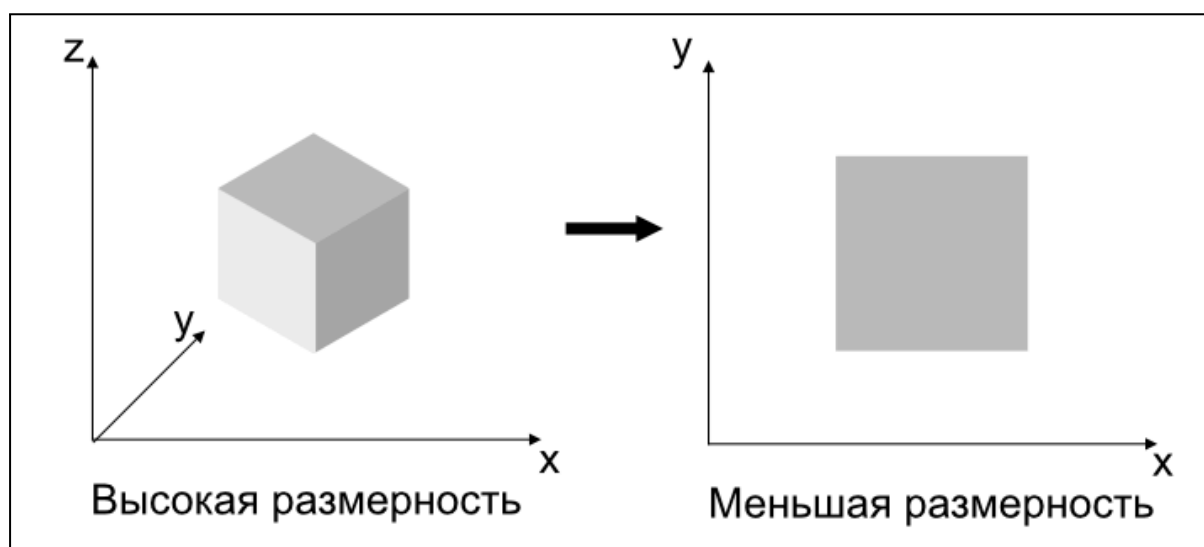


Рис. 1.9. Снижение размерности с трёх признаков до двух

Параметрическое и непараметрическое обучение

Метод проб и ошибок или вычисления и вероятность

Помимо деления на обучение с учителем и без учителя, алгоритмы также делят по двум другим признакам: параметрический и непараметрический. Эти признаки не взаимозаменяемы с предыдущими, и по ним алгоритмы можно разбить в четыре разных категории: с учителем или без, а также параметрические или непараметрические. Если, как говорилось выше, наличие или отсутствие учителя (обучающей выборки данных) определяет тип выявляемых закономерностей, то параметричность задает способ хранения результатов обучения и зачастую метод обучения. Параметрическая модель характеризуется наличием фиксированного числа параметров, тогда как непараметрическая модель имеет бесконечное число параметров (определяется данными).

Можно сравнить два вида моделей с двумя портными: первый портной шьёт только по установленным размерам S, M, L, XL, и примеряет на каждого клиента по размеру, пока не найдет подходящий — это параметрическая модель; второй портной измеряет каждого клиента и шьёт одежду сразу нужного размера - это непараметрическая модель. Первый портной сможет обслужить почти всех клиентов, но могут возникнуть проблемы, если кто-то не вписывается в шаблонные размеры; второй портной сможет обслужить всех, но ему придется запоминать размер каждого клиента. Параметрические модели основаны на методе проб и ошибок, тогда как в основе непараметрических моделей зачастую лежат вычисления.

Параметрическое обучение с учителем

Метод проб и ошибок с использованием регуляторов

Параметрические модели обучения с учителем — модели, имеющие фиксированное число параметров, которое не зависит от размера обучающей выборки. Данные обрабатываются согласно значениям этих параметров и преобразуются в прогнозы.

Например, вы создали модель, в которую загрузили историю матчей футбольной команды со всеми победами и поражениями. В результате работы алгоритм выдаёт некую вероятность победы (например, 80%). Затем, когда появится новая информация о матчах, в зависимости от входных данных параметры будут изменены, чтобы прогноз был более точным.

Значения параметров в процессе обучения можно зафиксировать, чтобы найти самую точную их конфигурацию и подогнать модель под нужный вариант работы. Почти все параметрические системы работают по принципу “проб и ошибок” из-за необходимости настройки параметров, несмотря на то, что это и не их формальное определение. Непараметрические системы работают иначе, так как в основе их работы лежат вычисления, и новые параметры появляются при условии обнаружения чего-то пригодного для использования в вычислениях.

Параметрическое обучение без учителя

Кластеризация с использованием регуляторов

В параметрическом обучении без учителя параметры используются для группировки данных. При таком способе параметры используются для установки целевых значений, которые будут приводить к наиболее точному распределению по группам. Модель обучения без учителя в качестве выходных данных выводит вероятность принадлежности конкретной точки данных к кластеру, и параметры позволяют скорректировать это распределение.

Непараметрическое обучение

Методы на основе вычислений

В непараметрическом обучении количество параметров не предопределено и зависит от данных. В зависимости от числа признаков, которые были выявлены в данных, будет изменено и число параметров. Это значит, что результаты таких алгоритмов определяются в первую очередь теми данными, которые будут использоваться, а не человеком, как было в параметрическом обучении. Непараметрические модели позволяют выявить еще некоторые зависимости ценой скорости и потребляемой алгоритмом памяти.

Обучающий, проверочный и тестовый наборы данных

Стоит знать, что наборы данных разделяют в зависимости от случая их использования. В начальном этапе построения модели используется *обучающий набор данных* (training dataset). Он содержит отмеченные примеры, на которых машина и обучается.

Далее, используется *проверочный набор данных* (validation dataset). Он содержит данные, которые проверяют точность прогнозов модели на этапе обучения и настраивают параметры для лучшей работы алгоритма.

Для оценки окончательной модели используется *тестовый набор данных* (test dataset).

Основные термины

- **Набор данных** — результаты наблюдений, на основе которых обучается алгоритм. Часто представляется в виде таблицы или матрицы.
- **Обучающий пример** — строка в таблице набора данных.
- **Признак** (обозначается x) — столбец в таблице набора данных. Синоним предиктора, переменной, входных данных, атрибута или ковариаты.
- **Цель** (обозначается y) — синоним результата, выхода, переменной отклика, зависимой переменной, метки (класса) и эталона.
- **Функция потерь** — функция, измеряющая ошибку между прогнозами модели и реальными данными.

На рисунке ниже представлен пример набора данных об ирисах с измерениями 150 цветков трех сортов.

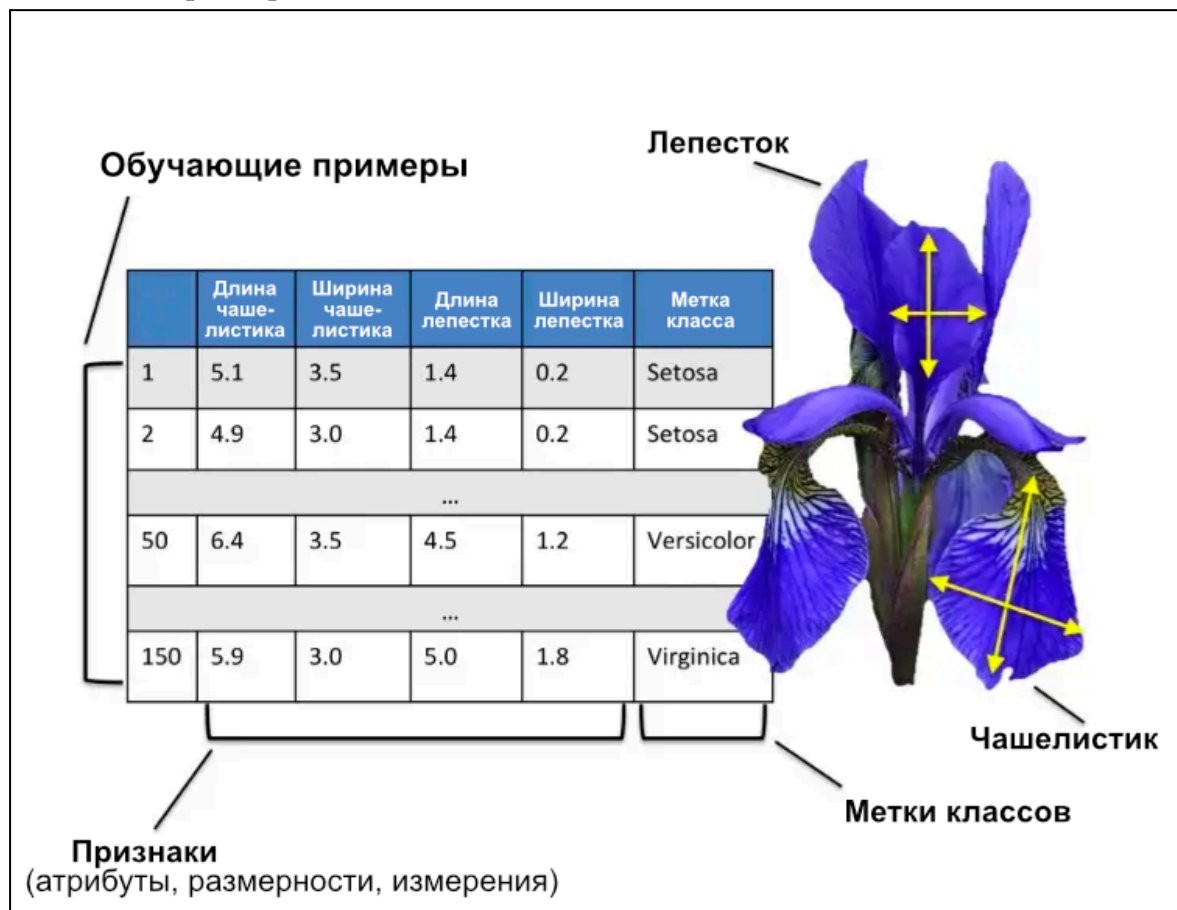


Рис. 1.10. Пример набора данных (Iris)

Глава 2. Инструменты

Установка Python и Jupyter Notebook

Python стал главным языком в поле машинного обучения по нескольким причинам:

- Во-первых, Python имеет простой синтаксис. Базовые знания английского языка могут помочь на интуитивном уровне понять принцип работы программы.
- Во-вторых, на Python существует множество библиотек, таких как, например, NumPy, Matplotlib и scikit-learn, которые будут использованы далее. Они позволяют создавать алгоритмы в десятки раз быстрее, пропуская ручное программирование всех методов.
- В-третьих, Python является стандартом индустрии и используется повсеместно. Как и в общении между людьми, ради простоты понимания друг друга стоит использовать знакомый обоим общий язык.

Стоит заметить, что в коде используется Python версии 3. Предыдущая версия 2 имеет множество отличий, и некоторый код может не работать. Имейте это ввиду при установке и использовании.

Установите Python с официального сайта python.org. В машинном обучении нам ещё понадобятся упомянутые выше библиотеки. Чтобы их установить, откройте командную консоль и выполните следующие команды. Строки, в начале которых есть символ #, — это комментарии, их вводить не нужно.

```
# 1. Проверка, что Python установлен
python -version
```

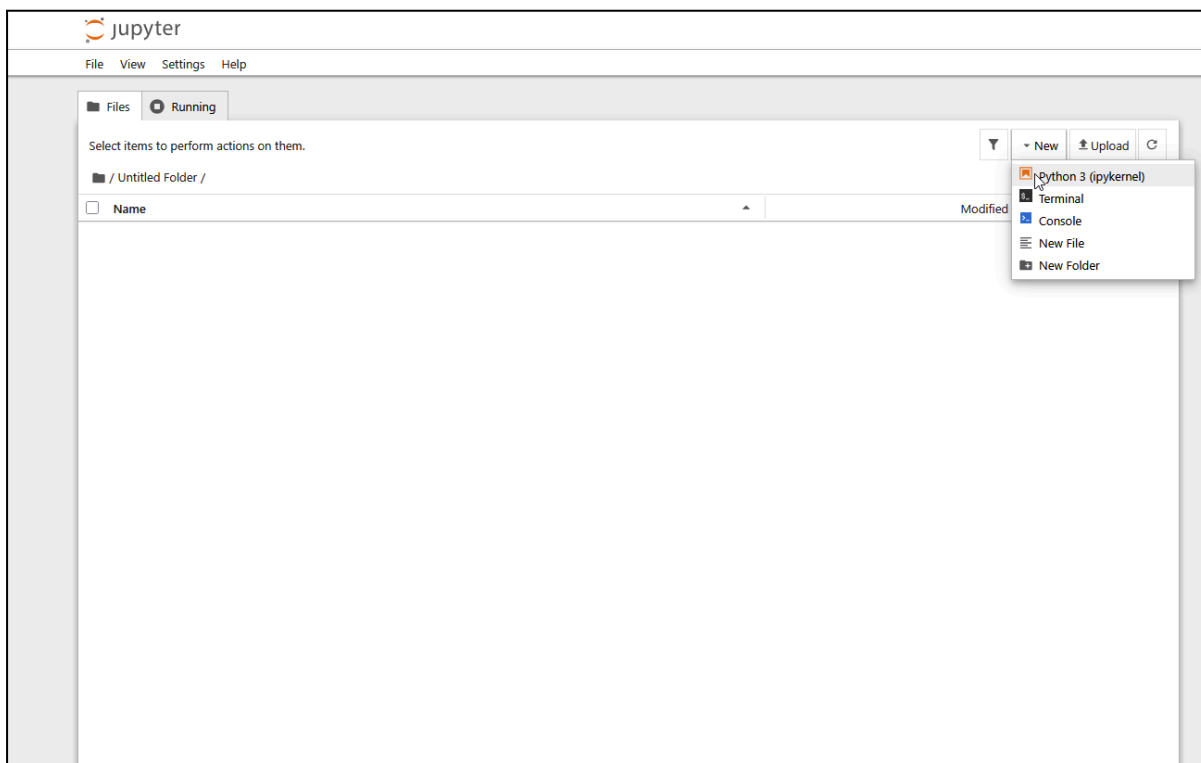
```
#2. Установка необходимых библиотек
pip install numpy matplotlib scikit-learn jupyter
```

```
#3. Запуск Jupyter Notebook
jupyter notebook
```

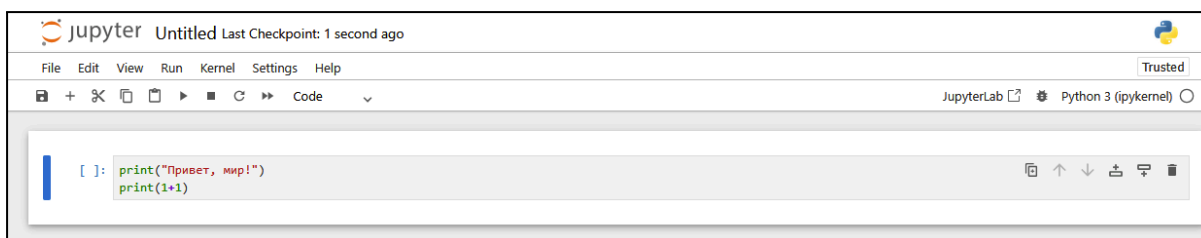
Будем использовать пакетный менеджер `pip`, идущий в комплекте с Python. После выполнения последней команды, в вашем браузере должна открыться вкладка Jupyter Notebook. Именно в этой среде благодаря удобству выполнения кода и наблюдения за процессом обучением модели мы и будем программировать.

Основы Jupyter Notebook

Впервые открыв Jupyter Notebook, вы увидите меню файлового менеджера. Для начала работы, создайте новое ядро (по сути, файл с кодом) Python и откройте его.



Вам откроется меню, содержащее пустую клетку. Впишите следующий код и проверьте его выполнение, используя кнопку запуска или сочетание клавиш Shift+Enter.



Если под клеткой вы видите текст, то всё успешно.

Теперь проверим, что все нужные библиотеки установлены. Выполните следующий код и проверьте, что результат, кроме номера версий, совпадает.

```
[3]: import numpy as np
import matplotlib.pyplot as plt
import sklearn

print("Версия NumPy:      ", np.__version__)
print("Версия scikit-learn:", sklearn.__version__)
print("Все библиотеки установлены!")
```

```
Версия NumPy:      2.4.5
Версия scikit-learn: 1.8.0
Все библиотеки установлены!
```

Основы NumPy

NumPy — основа научных вычислений в Python. Каждая библиотека машинного обучения так или иначе основана на этом модуле.

Python умеет хранить числа, но в связи со своей внутренней структурой, делает это очень медленно. Для оптимизации этого процесса используется NumPy. В отличие от Python, будем использовать массивы из библиотеки, потому что они:

- Гораздо быстрее благодаря использованию языка C в коде модуля
- Более лаконичны и позволяют добавить 1000 чисел прямо в одной строке вместо использования долгого цикла
- Позволяют работать с данными высокой размерности, матрицами, таблицами и наборами данных.

```
import numpy as np
# Список из Python, где каждый элемент возводится в квадрат через цикл
numbers = [1,2,3,4,5]
squared_list = [x**2 for x in numbers]
print("Результат списка:", squared_list)

# Массив Numpy, позволяющий сделать всё за одну операцию
arr = np.array([1,2,3,4,5])
squared_arr = arr ** 2
print("Результат массива:", squared_arr)
```

Результат вывода:

Результат списка: [1, 4, 9, 16, 25]

Результат массива: [1 4 9 16 25]

Ещё пример работы с массивами:

```
import numpy as np

# Создание массива
a = np.array([10, 20, 30, 40])

# Массив с равномерно распределенными целыми числами от 0 до 10 с шагом 2
b = np.arange(0, 10, 2)      # [0, 2, 4, 6, 8]

# Массив с равномерно распределенными пятью рациональными числами
c = np.linspace(0, 1, 5)     # [0.0, 0.25, 0.5, 0.75, 1.0]

# Массивы нулей и единиц
d = np.zeros(4)              # [0. 0. 0. 0.]
e = np.ones((2, 3))          # 2 строки, 3 столбца из 1.0

# Генерация случайных чисел
f = np.random.rand(5)        # 5 случайных рациональных чисел от 0 до 1
g = np.random.randn(5)       # 5 случайных чисел из нормального распределения

print(b)
print(c)
print(e)
```

Результат вывода:

```
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1.  ]
[[1. 1. 1.]
 [1. 1. 1.]]
```

NumPy позволяет применить математическую операцию к сразу всем элементам. Это называется *векторизацией*, и позволяет избегать написания циклов.

```
x = np.array([1.0, 2.0, 3.0, 4.0, 5.0])

print("Добавить 10:      ", x + 10)
print("Умножить на 3:", x * 3)
print("Квадратный корень:  ", np.sqrt(x).round(3))

# Операции между соответствующими элементами двух массивов
y = np.array([10, 20, 30, 40, 50])
print("x + y:", x + y)
print("x * y:", x * y)
```

Результат вывода:

```
Добавить 10:      [11. 12. 13. 14. 15.]
Умножить на 3: [ 3.  6.  9. 12. 15.]
Квадратный корень: [1.   1.414 1.732 2.   2.236]
x + y: [11. 22. 33. 44. 55.]
x * y: [ 10.  40.  90. 160. 250.]
```

Также стоит знать основные методы из статистики, которые могут понадобиться при анализе набора данных:

```
scores = np.array([72, 85, 91, 63, 78, 95, 88, 74, 66, 82])

print(f"Среднее значение: {np.mean(scores):.1f}")
print(f"Медианное значение: {np.median(scores):.1f}")
print(f"Стандартное отклонение: {np.std(scores):.1f}")
print(f"Минимальное значение: {np.min(scores)}")
print(f"Максимальное значение: {np.max(scores)}")
print(f"Размах: {np.max(scores) - np.min(scores)}")
```

Результат вывода:

```
Среднее значение: 79.4
Медианное значение: 80.0
Стандартное отклонение: 10.1
Минимальное значение: 63
Максимальное значение: 95
Размах: 32
```

Работа с отдельными элементами массивов устроена так же, как и в списках в Python.

```
data = np.array([10, 20, 30, 40, 50, 60])

print(data[0])      # Первый элемент:      10
print(data[-1])     # Последний элемент:    60
print(data[1:4])    # Элементы 1, 2, 3:    [20 30 40]
print(data[:,2])    # Каждый второй элемент: [10 30 50]

# Двумерные 2D массивы (матрицы) постоянно используются в МО
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print(matrix[1, 2])  # Строка 1, столбец 2:  6
print(matrix[:, 0])  # Все строки, столбец 0: [1 4 7]
print(matrix[0, :])  # Строка 0, все столбцы: [1 2 3]
```

Результат вывода:

```
10
60
[20 30 40]
[10 30 50]
6
[1 4 7]
[1 2 3]
```

Каждый массив имеет форму или же структуру. В scikit-learn важны формы, и признаки всегда должны представлять из себя 2D-массив, где строки — обучающие примеры, а столбцы — признаки. Цели всегда должны быть одномерным массивом.

```
a = np.array([1, 2, 3, 4, 5, 6])
print("Обычная форма:", a.shape)          # (6,) <- Одномерный массив

# Изменение формы на 2 строки и 3 столбца
b = a.reshape(2, 3)
print("Новая форма:", b.shape)             # (2, 3)
print(b)

# -1 указывает NumPy автоматически определить размерность массива
c = a.reshape(-1, 1)                       # 6 строки, 1 столбец
print("Форма через вектор:", c.shape)      # (6, 1)
```

Результат вывода:

```
Обычная форма: (6,)
Новая форма: (2, 3)
[[1 2 3]
 [4 5 6]]
Форма через вектор: (6, 1)
```

Попробуйте сами

1. Создайте массив из квадратов первых 10 чисел (1, 4, 9, ..., 100) через `np.arange` и оператор `**`.
2. Сгенерируйте 100 случайных чисел от 0 до 1. Рассчитайте их среднее значение и стандартное отклонение. Насколько среднее близко к 0.5?
3. Создайте 3x3 единичную таблицу через `np.eye(3)`. Где она может пригодиться?

Основы Matplotlib

“Золотым правилом” машинного обучения является вывод данных на картинке перед обучением модели. Поэтому нам пригодится Matplotlib — наиболее популярная библиотека для визуализации данных на графиках в Python. Будем использовать интерфейс pyplot, который работает как цифровая доска: сначала вы рисуете каждый отдельный элемент, а потом вызываете `plt.show()` для вывода результата.

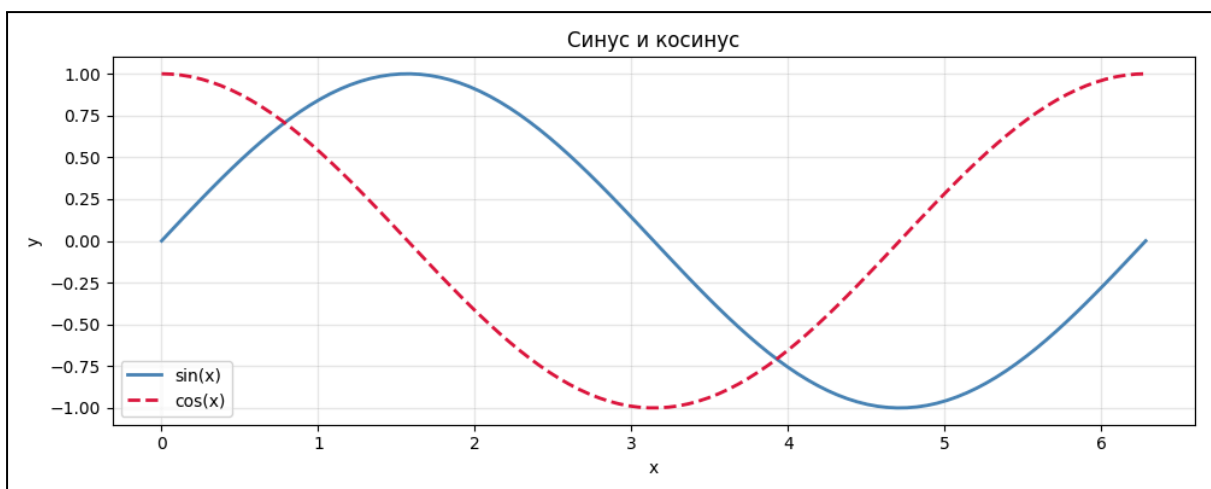
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100) # 100 точек от 0 до 2*pi

plt.figure(figsize=(10, 4)) # Размер выводимых фигур
plt.plot(x, np.sin(x), color="steelblue", linewidth=2, label="sin(x)")
plt.plot(x, np.cos(x), color="crimson", linewidth=2, linestyle="--",
label="cos(x)")
# 1. Вывод графика синуса цвета steelblue с толщиной линии 2 с подписью sin(x)
# 2. Вывод графика косинуса цвета crimson с толщиной пунктирной линии 2 с
подписью cos(x)

plt.xlabel("x") # Подпись к оси абсцисс
plt.ylabel("y") # Подпись к оси ординат
plt.title("Синус и косинус") # Подпись к графику сверху
plt.legend() # Вывод легенды - подписи к графику
plt.grid(alpha=0.3) # Вывод сетки с непрозрачностью 0.3
plt.tight_layout() # Вмещает график в границы
plt.show() # Вывод результата
```

Результат вывода:



Примечание: Цвета, типы линий и другие параметры могут быть найдены в документации библиотеки.

Отношение между двумя переменными показывают *диаграммы рассеяния*, или же *точечные графики*. Это наиболее вид графика в машинном обучении, используемый в анализе данных, представлении прогнозов модели и поиске закономерностей.

```
import matplotlib.pyplot as plt
import numpy as np

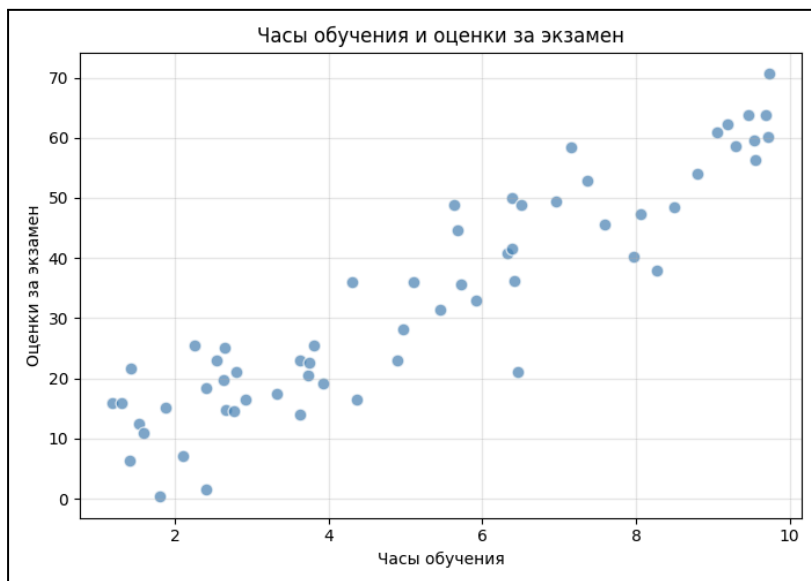
np.random.seed(42)

# Симуляция набора данных с часами обучения и оценками за экзамен
hours = np.random.uniform(1, 10, 60)
scores = 6.5 * hours + np.random.normal(0, 8, 60)

plt.figure(figsize=(7, 5))
plt.scatter(hours, scores,
            color="steelblue", alpha=0.7,
            edgecolors="white", s=60)

plt.xlabel("Часы обучения")
plt.ylabel("Оценки за экзамен")
plt.title("Часы обучения и оценки за экзамен")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Результат вывода:



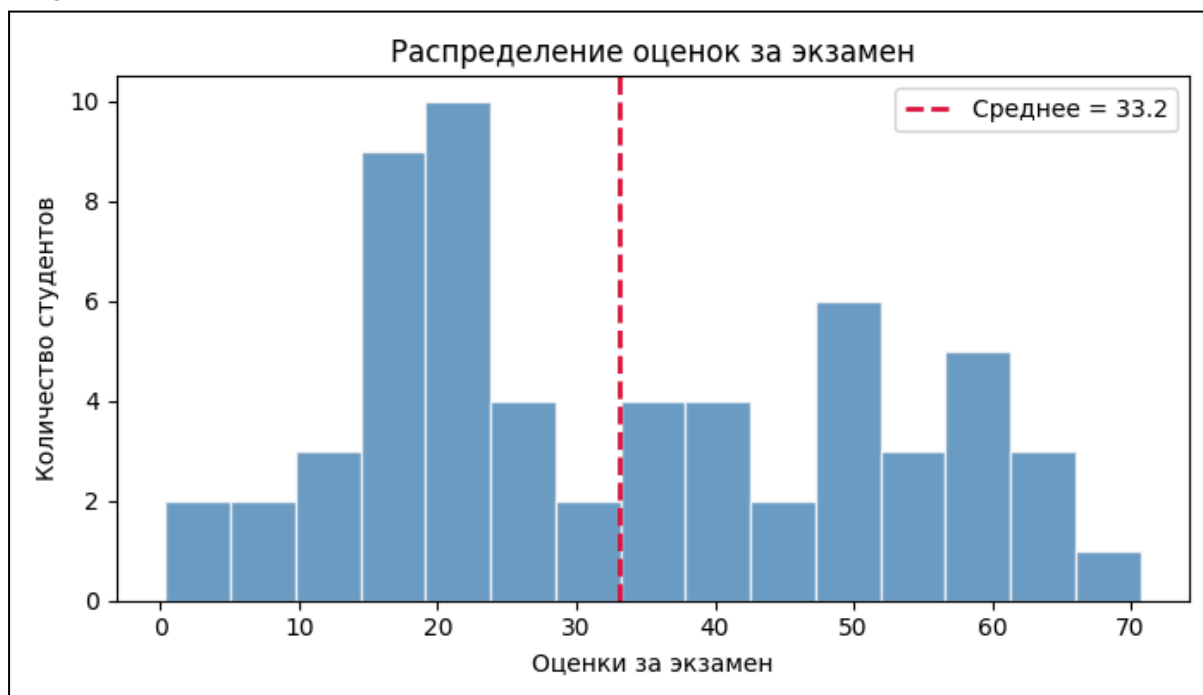
Гистограммы показывают распределение одной переменной — как много значений попадают в размах. Понимание распределений важно при создании модели.

```
plt.figure(figsize=(7, 4))
plt.hist(scores, bins=15, color="steelblue",
        edgcolor="white", alpha=0.8)

# Среднее отмечается вертикальной линией
plt.axvline(np.mean(scores), color="crimson",
        linestyle="--", linewidth=2,
        label=f"Среднее = {np.mean(scores):.1f}")

plt.xlabel("Оценки за экзамен")
plt.ylabel("Количество студентов")
plt.title("Распределение оценок за экзамен")
plt.legend()
plt.tight_layout()
plt.show()
```

Результат вывода:

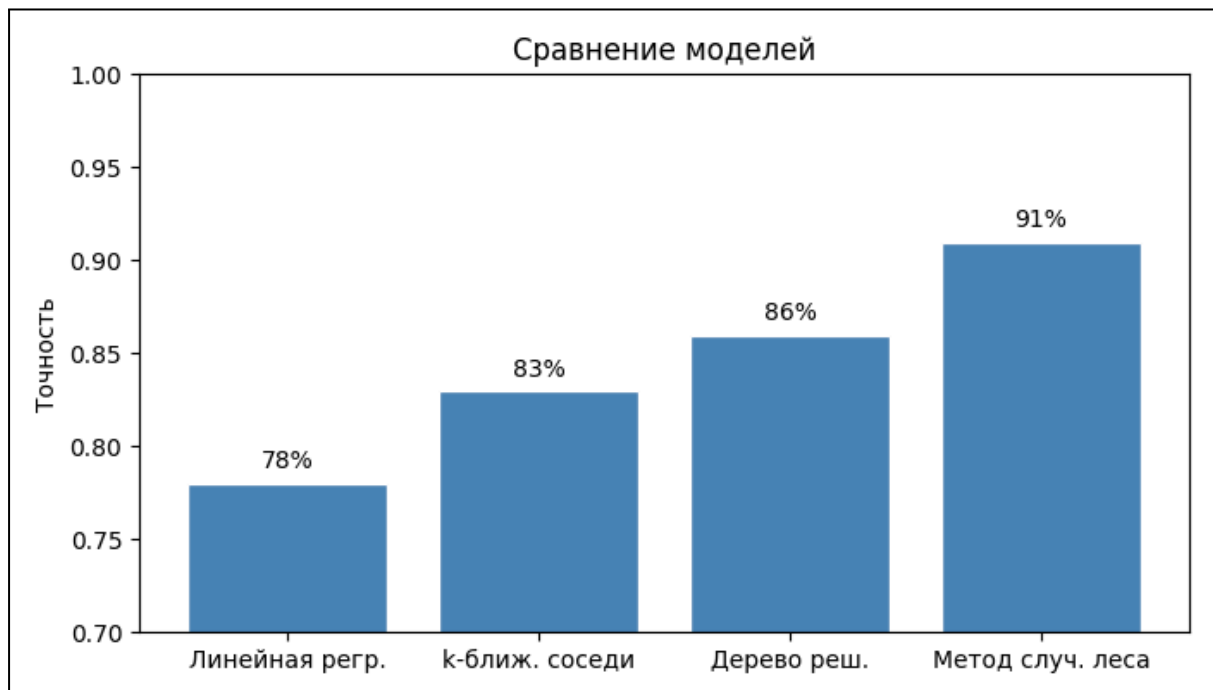


Пунктирная линия обозначает среднее значение. Распределение, похожее по форме на колокол называется *нормальным распределением* — важным аспектом реальных данных и статистики.

Сравнивать значения между категориями помогают *столбцовые диаграммы*. В МО вы будете использовать их, чтобы оценивать точность модели и визуализировать важность признаков.

```
model_names = ["Линейная регр.", "k-ближ. соседи", "Дерево реш.",  
               "Метод случ. леса"]  
accuracies = [0.78, 0.83, 0.86, 0.91]  
  
plt.figure(figsize=(7, 4))  
bars = plt.bar(model_names, accuracies,  
               color="steelblue", edgecolor="white")  
  
# Подпись каждого столбца с соответствующим значением  
for bar, acc in zip(bars, accuracies):  
    plt.text(bar.get_x() + bar.get_width() / 2,  
            bar.get_height() + 0.005,  
            f"{acc:.0%}",  
            ha="center", va="bottom", fontsize=10)  
  
plt.ylim(0.7, 1.0)  
plt.ylabel("Точность")  
plt.title("Сравнение моделей")  
plt.tight_layout()  
plt.show()
```

Результат вывода:



Метод `plt.subplots()` позволяет расположить несколько графиков в сетке. Он используется при сравнении признаков или моделей.

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

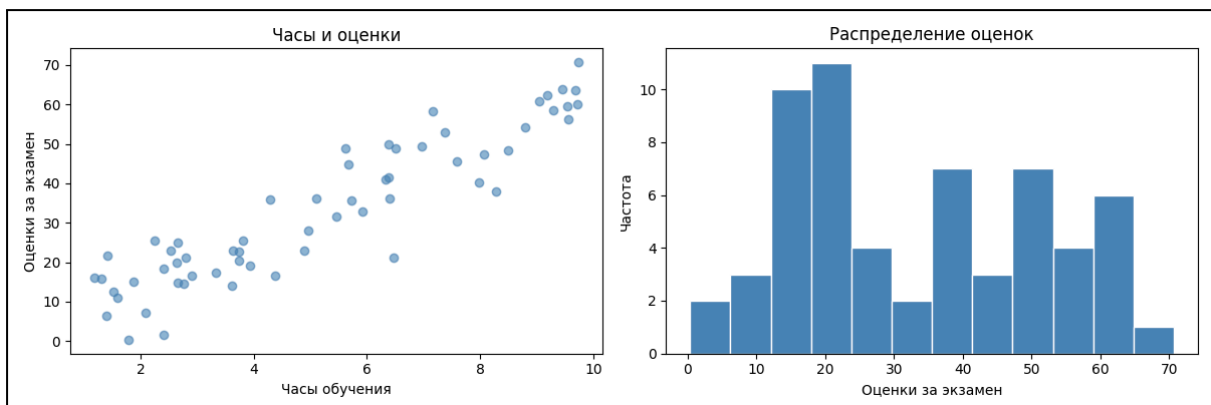
# Левая панель: диаграмма рассеяния
axes[0].scatter(hours, scores, color="steelblue", alpha=0.6)
axes[0].set_xlabel("Часы обучения")
axes[0].set_ylabel("Оценки за экзамен")
axes[0].set_title("Часы и оценки")

# Правая панель: гистограмма
axes[1].hist(scores, bins=12, color="steelblue", edgecolor="white")
axes[1].set_xlabel("Оценки за экзамен")
axes[1].set_ylabel("Частота")
axes[1].set_title("Распределение оценок")

plt.tight_layout()
plt.show()

```

Результат вывода:



Попробуйте сами

1. Поменяйте цвет диаграммы рассеяния на "coral" и параметр s= на 120. Что меняет это значение?
2. Отобразите графики $\sin(x)$, $\cos(x)$ и $\tan(x)$ на одном и нескольких графиках используя три разных цвета и стиля линий. Используйте `plt.ylim(-3, 3)` чтобы ограничить тангенс.
3. Сгенерируйте 1000 значений при помощи `np.random.randn(1000)`. Создайте гистограмму с 30 интервалами. Какую форму вы видите и как она называется?

Основы Scikit-learn и визуализации данных

Давайте объединим полученные знания и загрузим настоящий набор данных, проанализируем его при помощи NumPy и отобразим его через Matplotlib. Это точное повторение рабочего процесса настоящих проектов машинного обучения.

Scikit-learn предоставляет множество точных и готовых к использованию наборов данных для изучения и экспериментов. Их не нужно устанавливать отдельно, они были скачаны вместе с самой библиотекой ранее. Будем использовать ранее упомянутый набор данных Iris с измерениями 150 экземпляров трёх сортов цветков.

```
from sklearn.datasets import load_iris
import numpy as np
import matplotlib.pyplot as plt

iris = load_iris()

# iris.data -> двумерный массив NumPy, форма (150, 4)
# iris.target -> одномерный массив NumPy, форма (150,)
X = iris.data
y = iris.target

print("Форма набора данных:", X.shape)
print("Названия признаков:", iris.feature_names)
print("Метки классов: ", iris.target_names)
print("Первые три строки:\n", X[:3])
```

Результат вывода:

```
Форма набора данных: (150, 4)
Названия признаков: ['sepal length (cm)', 'sepal width (cm)', 'petal
length (cm)', 'petal width (cm)']
Метки классов:  ['setosa' 'versicolor' 'virginica']
Первые три строки:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]]
```

Перед выводом на график, рассчитаем некоторые статистические данные чтобы лучше понимать размер и распределение каждого признака.

```

print(f"Признак                Среднее    СтандОткл Минимум
Максимум")
print("-" * 55)

for i, name in enumerate(iris.feature_names):
    col = X[:, i]
    print(f"{name:<25} {np.mean(col):>6.2f} {np.std(col):>6.2f}\
{np.min(col):>6.2f} {np.max(col):>6.2f}")

```

Результат вывода:

Признак	Среднее	СтандОткл	Минимум	Максимум
sepal length (cm)	5.84	0.83	4.30	7.90
sepal width (cm)	3.06	0.43	2.00	4.40
petal length (cm)	3.76	1.76	1.00	6.90
petal width (cm)	1.20	0.76	0.10	2.50

Длина лепестков, petal length, имеет гораздо большее стандартное отклонение (1.76), чем ширина чашелистиков (0.44). Большой разброс часто означает более информативный признак, в чём мы и убедимся визуально.

Визуализация пар признаков рядом друг с другом — один из самых быстрых способов проанализировать новый набор данных. Раскрасим каждую точку в зависимости от сорта и посмотрим, можно ли разделить классы.

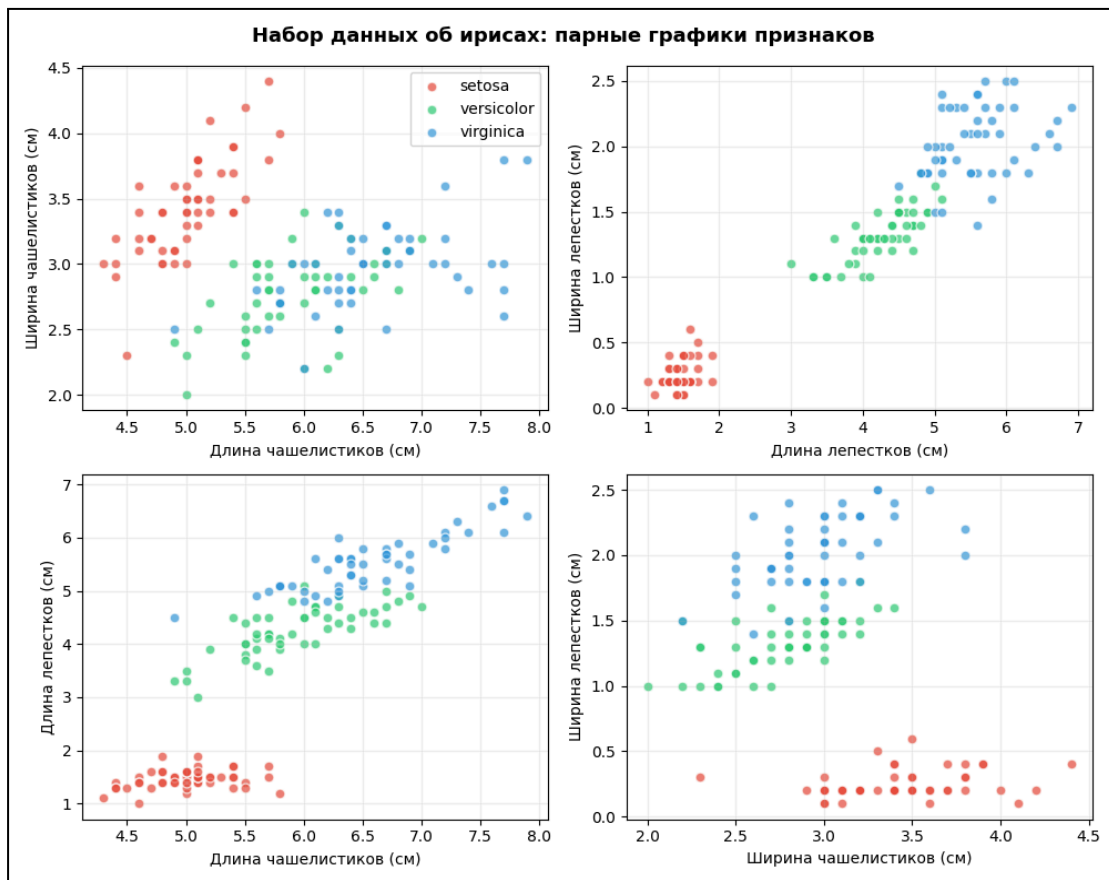
```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

pairs = [
    (0, 1, "Длина чашелистиков (см)", "Ширина чашелистиков (см)'),
    (2, 3, "Длина лепестков (см)", "Ширина лепестков (см)'),
    (0, 2, "Длина чашелистиков (см)", "Длина лепестков (см)'),
    (1, 3, "Ширина чашелистиков (см)", "Ширина лепестков (см)'),
]

for ax, (xi, yi, xlabel, ylabel) in zip(axes.flat, pairs):
    for cls in range(3):
        mask = y == cls
        ax.scatter(X[mask, xi], X[mask, yi],
                   color=colors[cls], alpha=0.7,
                   label=iris.target_names[cls],
                   edgecolors="white", s=40)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.2)

axes[0, 0].legend()
fig.suptitle("Набор данных об ирисах: парные графики признаков",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()
```

Результат вывода:



Можно заметить, что на втором графике, где сравниваются длина и ширина лепестков, все три сорта почти отделены друг от друга. Красные точки, обозначающие цветки сорта *setosa*, находятся в левом нижнем углу и имеют наименьшие лепестки; зелёные точки, обозначающие цветки сорта *versicolor*, находятся в центре; синие точки, обозначающие цветки сорта *virginica*, находятся в правом верхнем углу и имеют наибольшие лепестки. Таким образом график показывает, на какой признак стоит обратить внимание при классификации перед обучением модели.

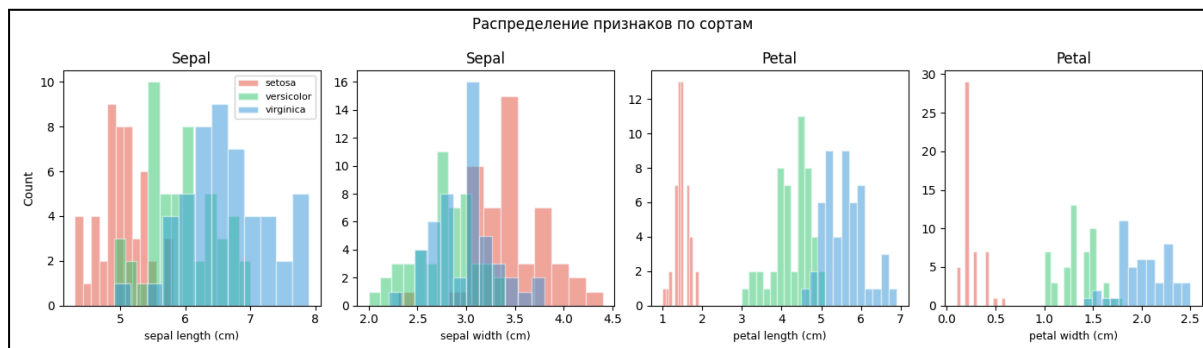
Пересекающиеся гистограммы показывают, различается ли распределение между классами. Если распределения пересекаются по большой площади - признак не так полезен для классификации.

```
fig, axes = plt.subplots(1, 4, figsize=(14, 4))

for i, (ax, feature) in enumerate(zip(axes, iris.feature_names)):
    for cls in range(3):
        ax.hist(X[y == cls, i], bins=12, alpha=0.5,
                color=colors[cls],
                label=iris.target_names[cls],
                edgecolor="white")
    ax.set_xlabel(feature, fontsize=9)
    ax.set_ylabel("Count" if i == 0 else "")
    ax.set_title(feature.split(" ")[0].capitalize())

axes[0].legend(fontsize=8)
fig.suptitle("Распределение признаков по сортам", fontsize=12)
plt.tight_layout()
plt.show()
```

Результат вывода:



Для длины и ширины чашелистиков три распределения явно пересекаются. Для длины и ширины лепестков они, напротив, практически отделены друг от друга, что пригодится в классификации по этим признакам.

В общем, анализ набора данных заключается в определённом порядке действий:

1. Загрузка набора данных и изучение его формы, названий признаков и меток классов
2. Вычисление базовых статистических данных по каждому признаку (среднее значение, стандартное отклонение, минимум, максимум)
3. Изображение раскрашенной диаграммы рассеяния для нахождения выделяемых кластеров
4. Изображение по-классовых гистограмм для определения информативных признаков
5. Выдвижение гипотезы: какие признаки важнее всего?

После этого процесса можно приступить к обучению модели. Замеченные детали помогут определить, какой алгоритм использовать, какие признаки сохранить и какую точность стоит ожидать.

По итогу этой главы, вы научились:

- Настраивать среду Python при помощи `pip`
- Создавать массивы через `NumPy` и управлять ими, в том числе и вычислять статистические данные
- Применять векторизированные операции, позволяющие обходить медленные циклы Python
- Изображать диаграммы рассеивания, гистограмм, столбцовые диаграммы и несколько графиков сразу при помощи `Matplotlib`
- Загружать набор данных из `scikit-learn` и следовать структурированному анализу перед обучением модели

Глава 3. Обучение с учителем

В первой главе мы ознакомились с понятием обучения с учителем и его частным случаем - регрессией. Теперь на основе этих знаний попробуем воссоздать это в коде.

Линейная регрессия

Как известно вам из школьной программы, прямая задаётся уравнением:

$$y = mx + b$$

- y — это вывод, или же наш прогноз
- m — это коэффициент изменения y за единицу x , или же наклон прямой
- b — это значение, в котором прямая пересекает ось ординат

Линейная регрессия помогает нам найти такие значения m и b , которые будут максимально точно описывать будущие прогнозы для набора данных.

Для каждой точки данных мы измеряем разность между прогнозом и действительным значением. Каждая разность возводится в квадрат, чтобы серьезные отклонения были более заметны, и после этого считается среднее значение среди этих квадратов. Алгоритм пытается настроить значения m и b до тех пор, пока *среднеквадратичное отклонение* не будет минимальным.

Теперь построим модель, которая предсказывает оценки за экзамен, основываясь на часах, проведённых за учебой. Создадим обучающий набор, обучим модель и выведем результат на графике.

Шаг 1. Импорт библиотек и создание набора данных

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(42)

# 50 учеников: часы обучения (1-10) и шумные значения оценок за экзамен
hours = np.random.uniform(1, 10, 50)
scores = 6 * hours + np.random.normal(0, 5, 50)

# scikit-learn требует двумерный массив признаков
X = hours.reshape(-1, 1)
y = scores
```

Шаг 2. Разделение на обучающий и тестовый наборы

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Тренировочные образцы: {len(X_train)}")
print(f"Тестовые образцы: {len(X_test)}")
```

Результат вывода:

```
Тренировочные образцы: 40
Тестовые образцы: 10
```

Шаг 3. Обучение модели

```
model = LinearRegression()
model.fit(X_train, y_train)

print(f"Наклон (m):      {model.coef_[0]:.2f}")
print(f"Пересечение с Oy (b): {model.intercept_:.2f}")
```

Результат вывода:

```
Наклон (m):      5.85
Пересечение с Oy (b): 0.77
```

В результате модель выявила, что каждый дополнительный час обучения соответствует 5.85 очкам, когда настоящий коэффициент был 6, что является вполне неплохим результатом.

Шаг 4. Оценка на тестовом наборе данных

```
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Среднее квадратичное отклонение: {mse:.2f}")
print(f"Точность: {r2:.2f}")
```

Результат вывода:

```
Среднее квадратичное отклонение: 19.91
Точность: 0.88
```

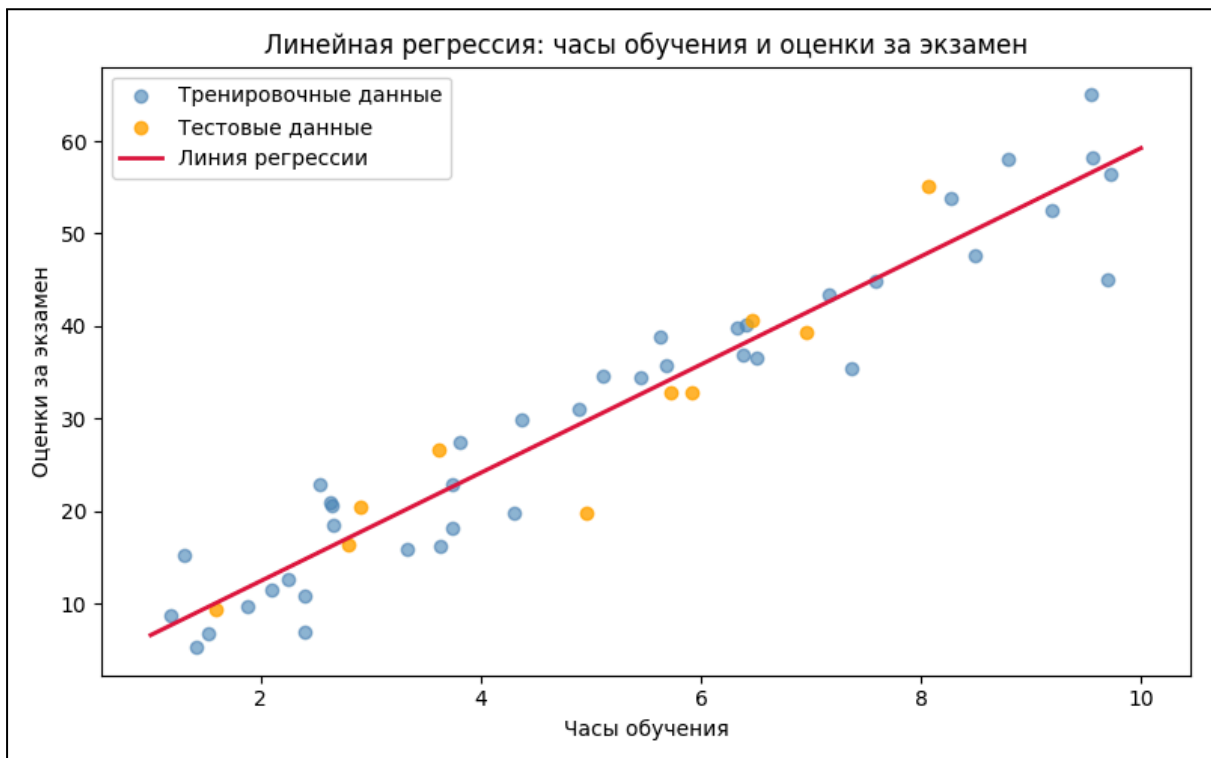
Шаг 5. Визуализация результата

```
plt.figure(figsize=(8, 5))
plt.scatter(X_train, y_train, color="steelblue", alpha=0.6,
label="Тренировочные данные")
plt.scatter(X_test, y_test, color="orange", alpha=0.8,
label="Тестовые данные")

x_line = np.linspace(1, 10, 100).reshape(-1, 1)
plt.plot(x_line, model.predict(x_line), color="crimson",
linewidth=2, label="Линия регрессии")

plt.xlabel("Часы обучения")
plt.ylabel("Оценки за экзамен")
plt.title("Линейная регрессия: часы обучения и оценки за экзамен")
plt.legend()
plt.tight_layout()
plt.show()
```

Результат вывода:



Синими точками обозначаются обучающие данные, оранжевыми - новые тестовые данные, а красными — ваша модель. Можно заметить, как линия проходит

прямо посреди распределения, не пересекая все точки, но находясь как можно ближе в среднем.

В реальных проблемах чаще всего более одного признака. Множественная линейная регрессия работает тем же самым образом, только требует сразу несколько наборов входных данных. В данном случае вы просто используете двумерный массив, где каждому столбцу соответствует признак.

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
X_multi = data.data      # 8 признаков у дома
y_prices = data.target    # медианная цена дома

X_tr, X_te, y_tr, y_te = train_test_split(
    X_multi, y_prices, test_size=0.2, random_state=42)

multi_model = LinearRegression()
multi_model.fit(X_tr, y_tr)
print(f"Точность в тестовом наборе: {multi_model.score(X_te, y_te):.2f}")
```

Результат вывода:

Точность в тестовом наборе: 0.58

Точность в 0.58 показывает, что модель описывает 58% разброса в ценах на жильё, что тоже неплохо для такого простого кода.

Попробуйте сами

1. Измените коэффициент в генерации данных до 10. Как изменился коэффициент у получившейся прямой?
2. Тренировка прошла на учениках, которые обучались только 4 и 8 часов. Улучшится ли точность модели?
3. Что означает точность 0 и 1? Может ли она быть отрицательной?

Среднеквадратичное отклонение (в англоязычной литературе называется MSE, mean squared error) является средним квадрата разницы между прогнозами и истинными значениями. Логично, что чем меньше это значение — тем лучше. Однако результатом является квадрат, так что часто из среднеквадратичного отклонения извлекают корень (в англоязычной литературе называют RMSE, root mean squared error), получая значение в изначальных единицах измерения.

```
import numpy as np
```

```

from sklearn.metrics import mean_squared_error

y_true = [50, 60, 70, 80]
y_pred = [48, 63, 68, 85]

mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
print(f"Среднеквадратичное отклонение: {mse:.2f}")
print(f"Корень среднеквадратичного отклонения: {rmse:.2f} (изначальные единицы измерения)")

```

Результат вывода:

Среднеквадратичное отклонение: 10.50

Корень среднеквадратичного отклонения: 3.24 (изначальные единицы измерения)

Под точностью имеется в виду так называемый *коэффициент детерминации*, или же R^2 (R-квадрат). Если задавать его формулой, то это будет выглядеть так:

$$R^2 = 1 - \frac{\text{Дисперсия ошибок модели (остаточная)}}{\text{Общая дисперсия данных}}$$

Дисперсия — это квадрат отклонений от среднего значения. R^2 показывает, насколько близки прогнозы модели к реальным значениям:

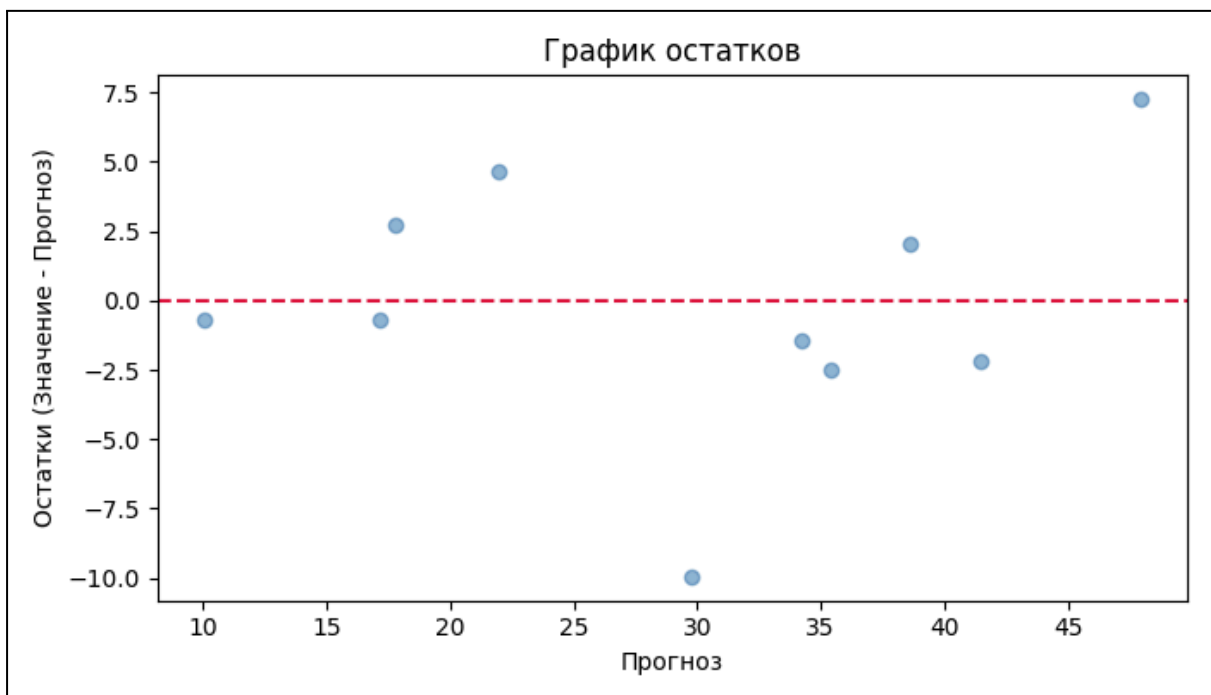
- $R^2 = 1.0$ — идеальные прогнозы
- $R^2 = 0.7$ — рабочая модель с описанными 70% разброса
- $R^2 = 0.0$ — не лучше, чем вычисление средних значений
- $R^2 < 0.0$ — хуже, чем вычисление средних значений. В таких случаях точно допущена ошибка и что-то пошло не так.

Чтобы показать разницу между истинным значением и прогнозом, используют *графики остатков*. Остатками, или же невязками, называют разницу прогнозируемых значений с истинными. Часто вывод их значений на графике может раскрыть дополнительную информацию, которую нельзя было бы узнать через суммарные метрики как R^2 или среднеквадратичное отклонение. Остатки должны выглядеть как случайный шум вокруг нуля, а не кривые.

```
# Используя прогнозы из начала главы
residuals = y_test - y_pred

plt.figure(figsize=(7, 4))
plt.scatter(y_pred, residuals, color="steelblue", alpha=0.6)
plt.axhline(y=0, color="crimson", linestyle="--")
plt.xlabel("Прогноз")
plt.ylabel("Остатки (Значение - Прогноз)")
plt.title("График остатков")
plt.tight_layout()
plt.show()
```

Результат вывода:



Если бы здесь была кривая или форма, похожая на веер, вероятно, пришлось бы использовать что-то кроме линейной регрессии.

Попробуйте сами

1. В примере с часами обучения, сформулируйте, что означает корень из среднеквадратичного отклонения? Какое значение по шкале от 0 до 100 будет лучшим?
2. Изобразите график остатков для набора данных California Housing, используемого выше. Есть ли какая-то закономерность, или же он случаен?
3. Как должен выглядеть идеальный график остатков? Возможно ли его достичь?

Классификация

Метод К ближайших соседей

Метод *К ближайших соседей* (KNN, K nearest neighbors) — наиболее интуитивный алгоритм классификации данных. Чтобы определить класс точки, рассматриваются К ближайших соседей (где К — произвольное число) в тренировочном наборе данных. В результате присваивается наиболее часто встречающийся класс. Ближайшие точки определяются по расстоянию, для определения которого есть несколько разных методов, но будем пользоваться стандартной Евклидовой метрикой, о которой вы уже знаете из школьной программы,

где расстояние между точками $A(x_1; y_1)$ и $B(x_2; y_2)$ равняется $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

- К = 1 — повторяет класс ближайшего соседа, очень чувствительно к шуму
- К = 5 — выбирает среди 5 соседей, более стабильно
- К = 20 — более плавная грань, но может упускать детали

Выбор числа К относится к задаче *оптимизации гиперпараметров*.

Гиперпараметр - это параметр, использующийся для управления процессом обучения. В данном случае от значения К зависит точность определяемых групп, и оно должно быть не слишком большим и не слишком малым.

Рассмотрим, как работает метод при К = 5 на примере набора данных с ирисами. Для начала, загрузим сам набор.

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
import numpy as np

iris = load_iris()
X, y = iris.data, iris.target

print("Признаки:", iris.feature_names)
print("Классы: ", iris.target_names)
print("Форма:  ", X.shape)
```

Результат вывода:

```
Признаки: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
Классы:  ['setosa' 'versicolor' 'virginica']
Форма:   (150, 4)
```

Теперь воспользуемся методом К ближайших соседей и сразу оценим его точность.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
print(f"Точность: {accuracy_score(y_test, y_pred):.2%}")
```

Результат вывода:

Точность: 100.00%

На этом наборе данных идеальная точность, благодаря тому, что он аккуратно разделён. Реальные наборы сложнее, но структура кода остаётся той же.

Давайте выведем границу разделения классов на графике.

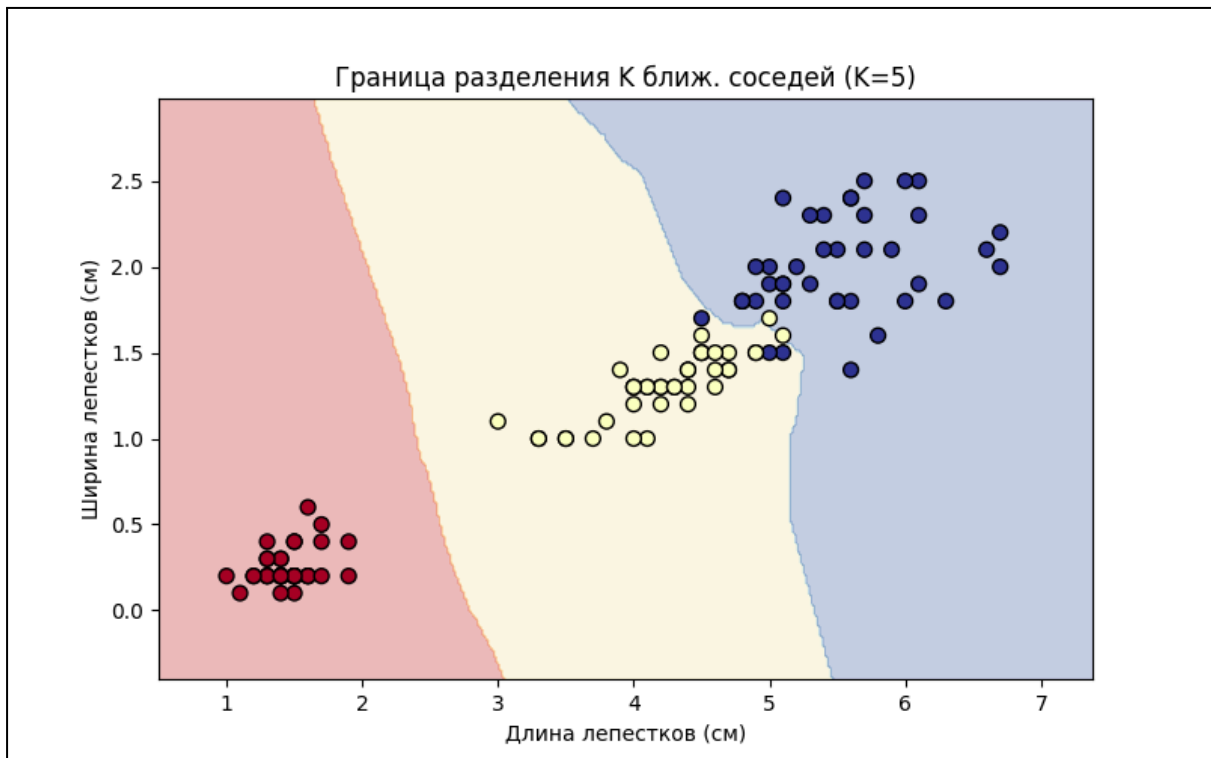
```
# Используем только длину и ширину лепестков для двумерного графика
X2 = X[:, 2:]
X_tr2, X_te2, y_tr2, y_te2 = train_test_split(
    X2, y, test_size=0.2, random_state=42)

knn2 = KNeighborsClassifier(n_neighbors=5)
knn2.fit(X_tr2, y_tr2)

# Форма для заливки фона
x_min, x_max = X2[:,0].min()-0.5, X2[:,0].max()+0.5
y_min, y_max = X2[:,1].min()-0.5, X2[:,1].max()+0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))
Z = knn2.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.figure(figsize=(8, 5))
plt.contourf(xx, yy, Z, alpha=0.3, cmap="RdYlBu")
plt.scatter(X_tr2[:,0], X_tr2[:,1], c=y_tr2,
            cmap="RdYlBu", edgecolors="k", s=50)
plt.xlabel("Длина лепестков (см)")
plt.ylabel("Ширина лепестков (см)")
plt.title("Граница разделения К ближ. соседей (K=5)")
plt.show()
```


Результат вывода:



Цветной фон показывает, какой в каждой точке прогнозируется класс. Края границ резкие, потому что метод рассматривает только расположение точек без сглаживания значений.

Рассмотрим, как влияет значение K на результат.

```
k_range      = range(1, 31)
test_scores  = []

for k in k_range:
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_train, y_train)
    test_scores.append(accuracy_score(y_test, knn_k.predict(X_test)))

plt.figure(figsize=(8, 4))
plt.plot(k_range, test_scores, marker="o", color="steelblue")
plt.xlabel("K")
plt.ylabel("Оценка точности")
plt.title("Точность в зависимости от K")
plt.grid(alpha=0.3)
plt.show()
```

Результат вывода:



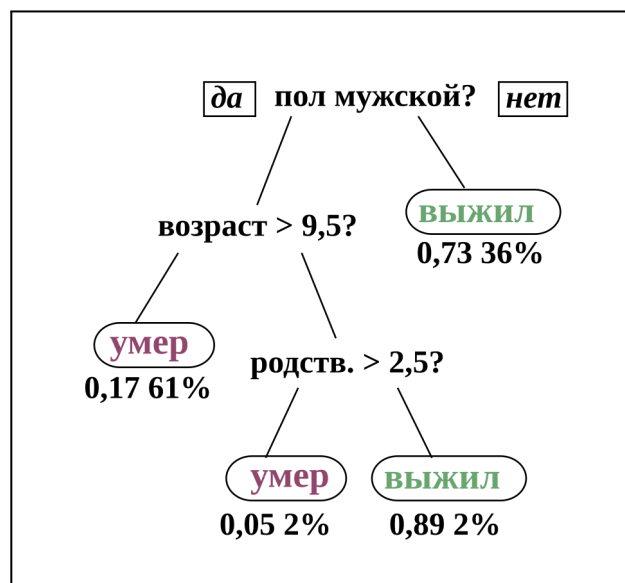
Как видите, маленькие и большие K образуют провалы в точности. Подходят средние K равные 5 или 10. Попробуйте избегать нечётных значений K, чтобы не сталкиваться с ничьёй в количестве соседей разных классов.

Попробуйте сами

1. Попробуйте K = 1, K = 10 и K = 50. Как от этого меняется форма границы и точность?
2. Используйте все четыре признака ирисов. Увеличивается ли точность?
3. Почему метод K ближайших соседей работает медленно на больших наборах данных?

Деревья принятия решений

Деревья принятия решений — ещё один метод классификации. Алгоритм каждый раз отделяет по одному признаку. Каждый этап программа задаёт вопрос: “Какой признак и какая граница лучше разделяет классы?”, разделяя данные на две группы. Так продолжается до тех пор, пока каждая категория не станет достаточно точной или маленькой. Такие алгоритмы называют деревьями, потому что каждый вопрос рождает две



“ветви” — признаки, а ответы на него образуют “листья” — классы.
Давайте проверим этот алгоритм на том же наборе данных, что и раньше.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# max_depth контролирует, чтобы алгоритм не разбивал данные на слишком
# маленькие группы.
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

y_pred = tree.predict(X_test)
print(f"Точность: {accuracy_score(y_test, y_pred):.2%}")
```

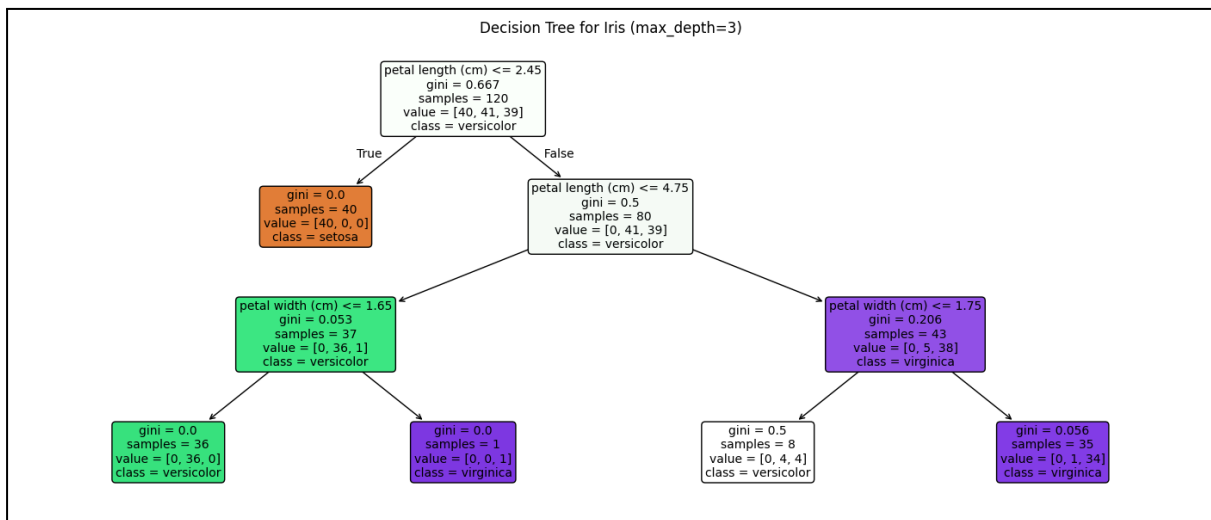
Результат вывода:

Точность: 100.00%

Сразу же визуализируем дерево.

```
plt.figure(figsize=(14, 6))
plot_tree(tree,
          feature_names=iris.feature_names,
          class_names=iris.target_names,
          filled=True,
          rounded=True,
          fontsize=10)
plt.title("Дерево решений (max_depth=3)")
plt.tight_layout()
plt.show()
```

Результат вывода:



В каждой ячейке мы можем увидеть критерий разделения (задаваемый вопрос), количество образцов, которое прошло и само разделение по классам. В самом низу оказались “листья” — финальные прогнозы. Возможность увидеть каждый задаваемый вопрос является большим преимуществом деревьев принятия решений.

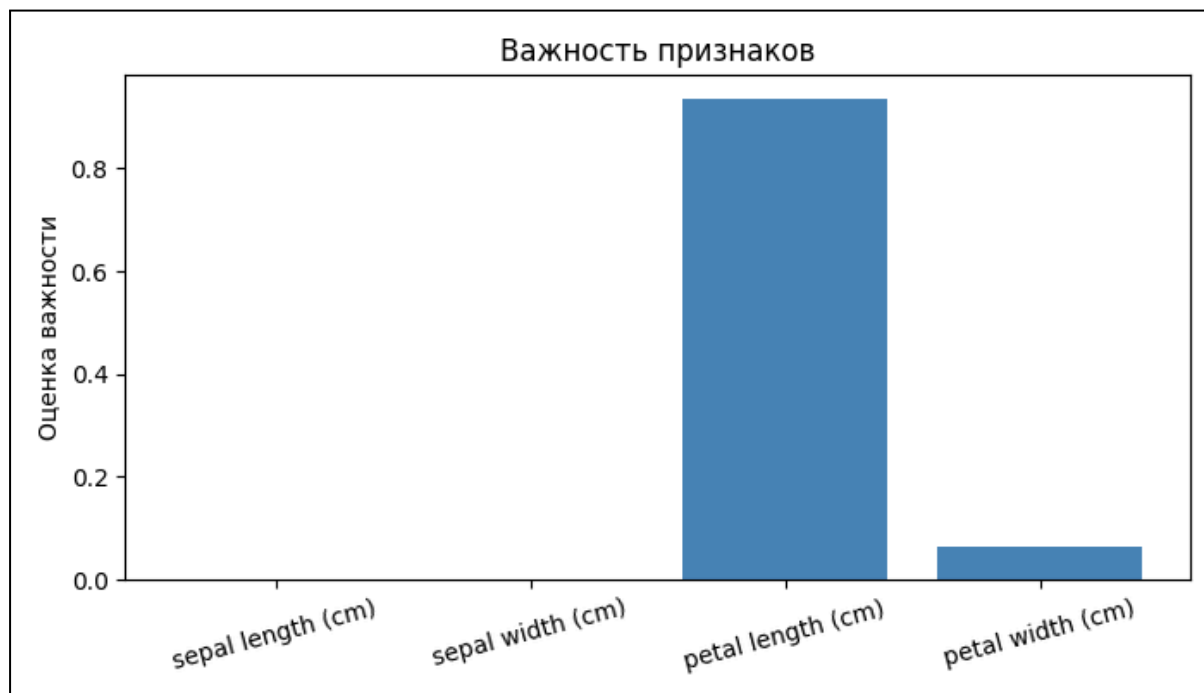
Оценим также важность признаков в классификации.

```
importances = tree.feature_importances_

plt.figure(figsize=(7, 4))
plt.bar(iris.feature_names, importances, color="steelblue")
plt.ylabel("Оценка важности")
plt.title("Важность признаков")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()

for name, score in zip(iris.feature_names, importances):
    print(f"{name:<30} {score:.3f}")
```

Результат вывода:



Видно, что данные по чашелистикам не приносят никакой информации в отличие от данных по лепесткам. Важность признаков помогает вам лучше понимать ваши данные и избегать бесполезные элементы.

Не имея предела, дерево принятия решений будет продолжать разделять данные до момента, пока в точности не повторит изначальный набор данных, проделав операцию для каждой точки. Прогнозы такого алгоритма не будут нести никакой ценности. Для этого используется параметр `max_depth`, контролирующий количество разделений, или же глубину дерева.

Попробуйте сами

1. Установите `max_depth = 1`. Какой вопрос задаёт дерево? Является ли он наиболее важным?
2. Установите `max_depth = None`. Что происходит с точностью данных?
3. Попробуйте дерево решений на искусственном наборе данных с часами обучения и оценками за экзамен из предыдущих частей. Как он выглядит в сравнении с линейной моделью?

По итогу этой главы, вы научились:

- Использовать и визуализировать алгоритмы линейной регрессии, метода К ближайших соседей и дерева решений

Глава 4. Обучение без учителя

Метод К-средних

Метод К-средних (K-means) — наиболее популярный метод кластеризации. Он разбивает набор данных на К кластеров, где каждая точка принадлежит к кластеру, чей *центроид* (центр) лежит ближе всего к ней. Вы выбираете К, а алгоритм сам определяет положение центроидов.

Алгоритм действует по порядку:

1. Расставляет К центроидов случайно в пространстве признаков
2. Присваивает каждой точке метку со значением ближайшего центроида
3. Каждый центроид движется к средней позиции между распределенными к нему точками
4. Шаги 2 и 3 повторяются до тех пор, пока центроиды не определяют точное положение и не перестанут двигаться (метод К-средних всегда сходится, то есть финальное расположение центроидов гарантированно существует)

Простота алгоритма является как и его главным преимуществом, так и большим минусом.

Сгенерируем набор данных с тремя искусственными кластерами (представим, что не знаем, какими именно) и используем метод К-средних, чтобы определить их.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

np.random.seed(42)

# Создадим 300 точек с тремя кластерами
X, y_true = make_blobs(n_samples=300, centers=3,
                       cluster_std=0.8, random_state=42)

print("Форма данных:", X.shape)
print("Истинные метки (для перепроверки):", np.unique(y_true))
```

Результат вывода:

Форма данных: (300, 2)

Истинные метки (для перепроверки): [0 1 2]

Будем использовать К-средние с K=3.

```
# n_init=10 означает, что алгоритм будет запущен 10 раз со случайными
# стартовыми точками и будет выбран лучший результат с меньшим
# количеством движения центроидов
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
kmeans.fit(X)

# Метки кластеров, присвоенные каждой точке
labels = kmeans.labels_

# Финальные позиции центроидов
centroids = kmeans.cluster_centers_

print("Метки кластеров (первые 10):", labels[:10])
print("Позиции центроидов:\n", centroids.round(2))
```

Результат вывода:

Метки кластеров (первые 10): [1 1 0 2 1 2 0 2 0 0]

Позиции центроидов:

[[-2.61 9.04]

[-6.88 -6.96]

[4.73 2.]]

Также визуализируем результат.

```

colors = ["#E74C3C", "#2ECC71", "#3498DB"]

plt.figure(figsize=(8, 6))

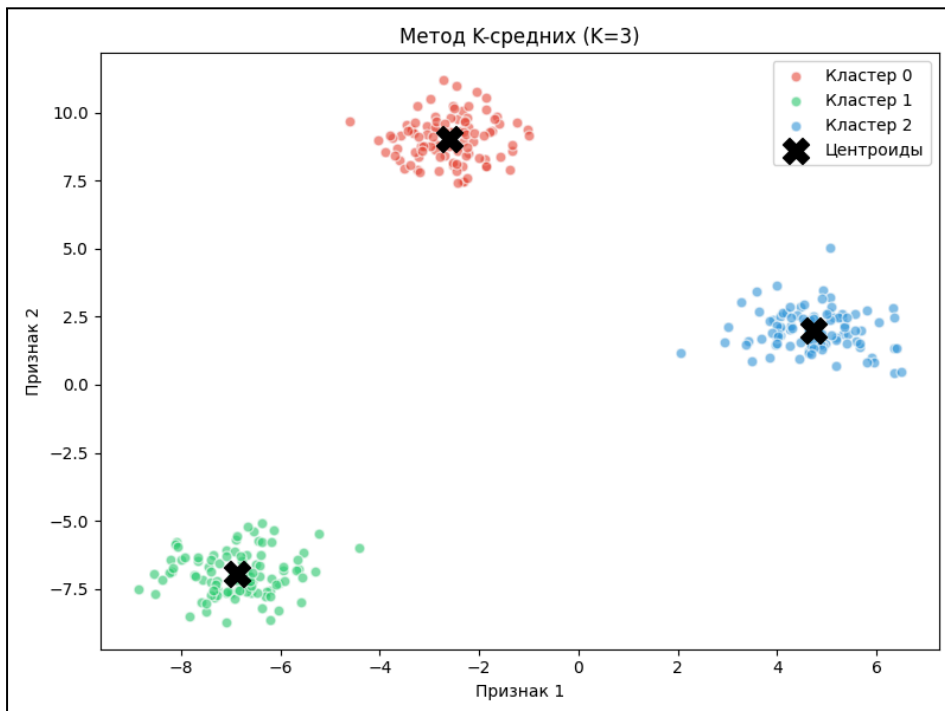
# Расположим точки по их назначенным кластерам
for k in range(3):
    mask = labels == k
    plt.scatter(X[mask, 0], X[mask, 1],
                color=colors[k], alpha=0.6,
                edgecolors="white", s=40,
                label=f"Кластер {k}")

# Изобразим центроиды как большие чёрные X
plt.scatter(centroids[:, 0], centroids[:, 1],
            marker="X", s=250, color="black",
            zorder=5, label="Центроиды")

plt.xlabel("Признак 1")
plt.ylabel("Признак 2")
plt.title("Метод К-средних (K=3)")
plt.legend()
plt.tight_layout()
plt.show()

```

Результат вывода:



Три раскрашенных кластера, центроид которых находится посередине. Метод К-средних определил естественные группы не зная изначальных меток.

Как только алгоритм был обучен, К-средние могут присваивать каждую новую точку к ближайшему центроиду используя метод `predict()`.

```
# Прогноз кластера для новых трёх точек
new_points = np.array([[0.0, 0.5], # Где-то посередине
                       [2.5, 1.0], # Рядом с кластером 0
                       [0.0, -2.5]]) # Рядом с кластером 2

predictions = kmeans.predict(new_points)
print("Спрогнозированные кластеры:", predictions)

for point, cluster in zip(new_points, predictions):
    print(f" Точка {point} -> Кластер {cluster}")
```

Результат вывода:

```
Спрогнозированные кластеры: [2 2 2]
Точка [0.  0.5] -> Кластер 2
Точка [2.5 1. ] -> Кластер 2
Точка [ 0. -2.5] -> Кластер 2
```

Попробуйте сами

1. Измените значение `n_clusters` на 5 и запустите код снова. Естественно ли выглядит результат или же оригинальные точки разделены лишним образом?
2. Попробуйте использовать `make_blobs` со значением `cluster_std=2.5` (более шумные кластеры). Как с этим справляется метод К-средних? Чисто ли выглядит результат?
3. Что произойдёт, если не установить значение `random_state`? Запустите алгоритм несколько раз. Каждый ли раз получаются те же самые кластеры? Почему это так?

Использование метода К-средних требует выбора числа К перед запуском. Но как в случае реальных данных мы сможем узнать количество кластеров? Метод К-средних следит за параметром, называемым *инерцией* (`inertia`). Это значение равно сумме квадратов расстояний от каждой точки до назначенного ей центроида. Чем меньше инерция — тем меньше кластеры.

```

inertia_values = []
K_range = range(1, 11)

for k in K_range:
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    km.fit(X)
    inertia_values.append(km.inertia_)

print("K -> Инерция")
for k, inertia in zip(K_range, inertia_values):
    print(f" K={k}: {inertia:.1f}")

```

Результат вывода:

```

K -> Инерция
K=1: 20120.5
K=2: 5526.5
K=3: 362.8
K=4: 318.1
K=5: 273.4
K=6: 233.6
K=7: 201.0
K=8: 172.9
K=9: 150.0
K=10: 139.3

```

И если мы изобразим график отношения K к инерции, мы увидим резкий спад, за которым следует медленное выпрямление. Точку смены называют “локтем” и она является хорошим приближением к настоящему числу кластеров.

```

plt.figure(figsize=(8, 4))
plt.plot(K_range, inertia_values, marker="o",
         color="steelblue", linewidth=2)

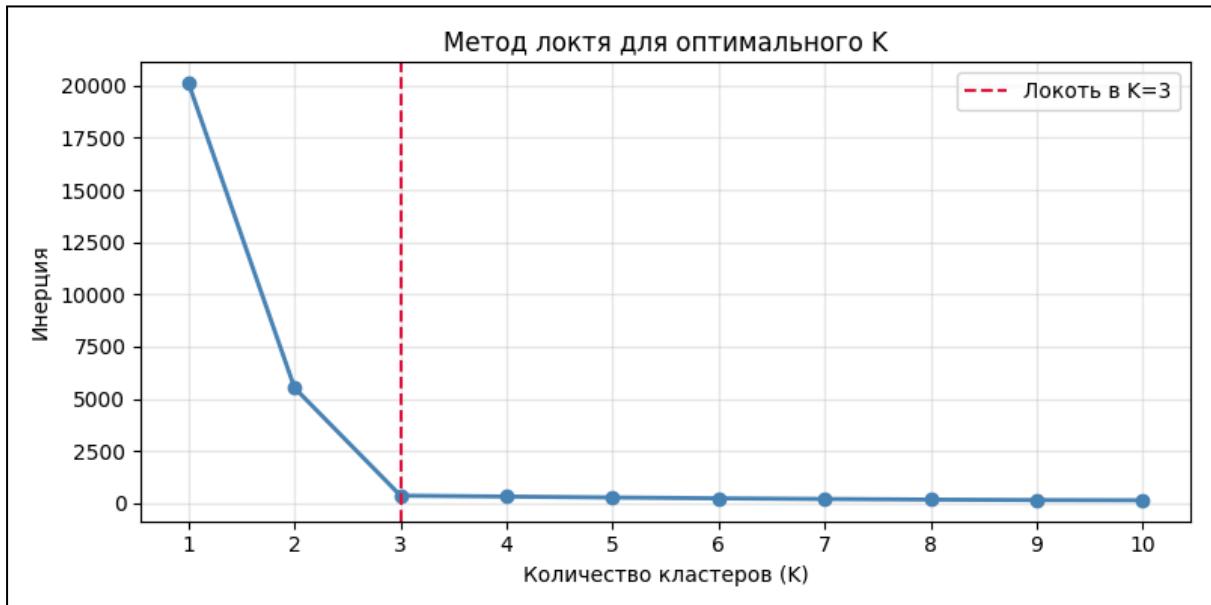
# Highlight the elbow at K=3
plt.axvline(x=3, color="crimson", linestyle="--",
           linewidth=1.5, label="Локоть в K=3")

plt.xlabel("Количество кластеров (K)")
plt.ylabel("Инерция")
plt.title("Метод локтя для оптимального K")
plt.xticks(K_range)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()

```

```
plt.show()
```

Результат вывода:



Можно увидеть, как с $K=1$ до $K=3$ значение инерции резко падает, а после этого практически не изменяется. Это означает, что добавление новых кластеров никак не улучшает результат. Таким образом, $K=3$ подсказывает, что всего кластеров три. К слову, “локоть” не всегда очевиден, и на реальных данных график может быть сложнее. В таких случаях используется оценка *силуэта*.

Оценка силуэта измеряет для каждой точки два значения: насколько близко она расположена к назначенному кластеру и как далеко она от остальных кластеров. Результат представляет собой число от -1 до +1, где -1 означает, что точка была отнесена к неправильному кластеру, 0 означает что точка расположена на границе между двумя кластерами и +1 означает лучший сценарий, где точка идеально расположена близко к своему кластеру и далеко от других.

```

from sklearn.metrics import silhouette_score

sil_scores = []

for k in range(2, 11): # силуэт требует как минимум 2 кластера
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels_k = km.fit_predict(X)
    sil_scores.append(silhouette_score(X, labels_k))

plt.figure(figsize=(8, 4))
plt.plot(range(2, 11), sil_scores, marker="o",
         color="steelblue", linewidth=2)

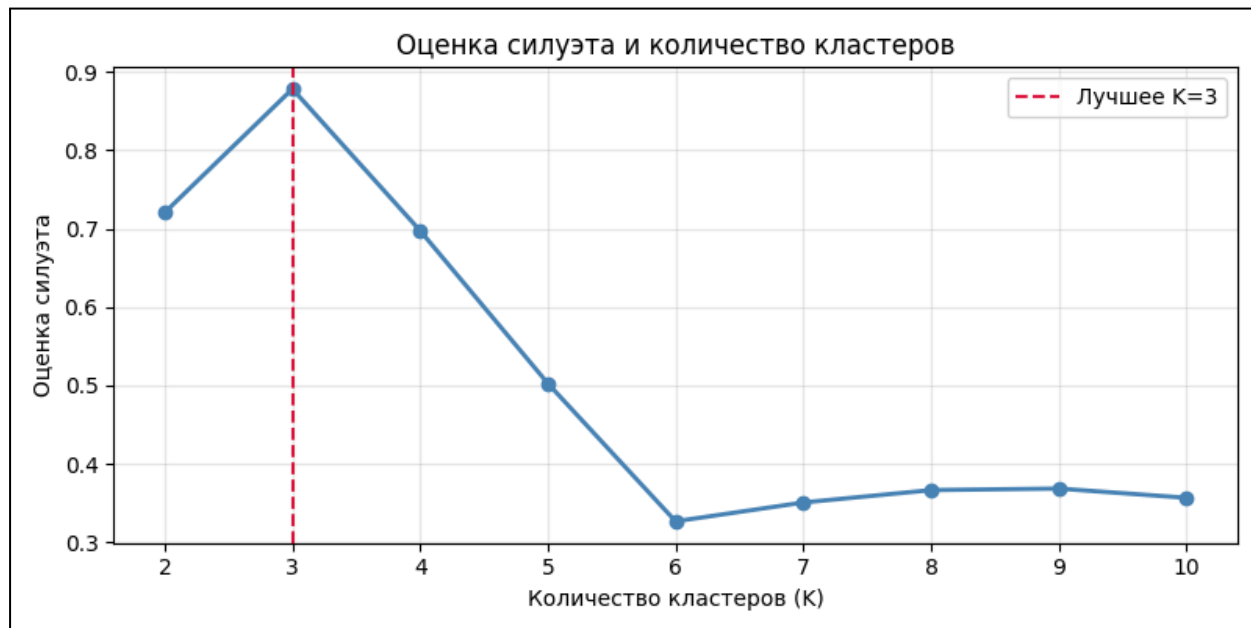
best_k = range(2, 11)[sil_scores.index(max(sil_scores))]
plt.axvline(x=best_k, color="crimson", linestyle="--",
           linewidth=1.5, label=f"Лучшее K={best_k}")

plt.xlabel("Количество кластеров (K)")
plt.ylabel("Оценка силуэта")
plt.title("Оценка силуэта и количество кластеров")
plt.xticks(range(2, 11))
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Лучшее K по силуэту: {best_k}")
print(f"Лучшая оценка: {max(sil_scores):.3f}")

```

Результат вывода:



Лучшее K по силуэту: 3

Лучшая оценка: 0.878

Оценка силуэта в 0.878 при K=3 — отличный результат. Метод “локтя” и оценка силуэта дали один и тот же результат, что является крепким доказательством, что три — правильное значение кластеров в этом наборе данных.

Попробуйте сами

1. Сгенерируйте данные, используя `make_blobs(centers=5)`, но не указывайте в методе K-средних, сколько есть кластеров. Используйте метод “локтя” и оценку силуэта для нахождения K. Сходятся ли результаты между собой?
2. Какая оценка силуэта достаточно хороша для настоящих данных? Существует ли точное значение, начиная с которого можно уверенно утверждать о правильности результата?
3. Используйте метод “локтя” на наборе данных Iris. Являются ли результатом три кластера, соответствующих трём известным сортам?

Кластеризация используется повсеместно, начиная от алгоритмов рекомендаций в социальных сетях, заканчивая сжатием изображений и документов. При этом используются и другие методы помимо метода K-средних. Например, когда мы не знаем значение кластеров и метод “локтя” и оценка силуэта выдают разные значения. Тем не менее, обучение без учителя и в частности метод K-средних — незаменимые инструменты при работе в области машинного обучения.

По итогу этой главы, вы научились:

- Использовать обучение без учителя на ещё не отмеченных данных
- Использовать Метод K-средних для разбиения данных по K кластерам

- Прогнозировать значение K используя метод “локтя” и оценку силуэта

СПИСОК ИСТОЧНИКОВ:

1. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani. An Introduction to Statistical Learning (with Applications in Python, Second Edition), 2023;
2. Машинное обучение. MachineLearning.ru— профессиональный информационно-аналитический ресурс, посвященный машинному обучению, распознаванию образов и интеллектуальному анализу данных. [Электронный ресурс] URL: http://www.machinelearning.ru/wiki/index.php?title=Машинное_обучение (дата обращения 10.05.2026)
3. Mitchell Tom. Machine Learning, 1997;
4. Aurélien Géron. Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow. Concepts, Tools and Techniques to Build Intelligent Systems, 2019;
5. Траск Эндрю. Грокаем глубокое обучение. — СПб.: Питер, 2019. — 352 с.: ил. — (Серия «Библиотека программиста»).
6. Рашка, С. Машинное обучение с PyTorch и Scikit-Leam: Пер. с англ. / С. Рашка, Ю. Лю, В. Мирджалили. - Астана: Фолиант, 2024. - 688 с.: ил.